

Logic Design and Switching Theory

Lecture – 8

Subtitle: Decoders, MUX & DMUX

by
Ms. Hera Noor

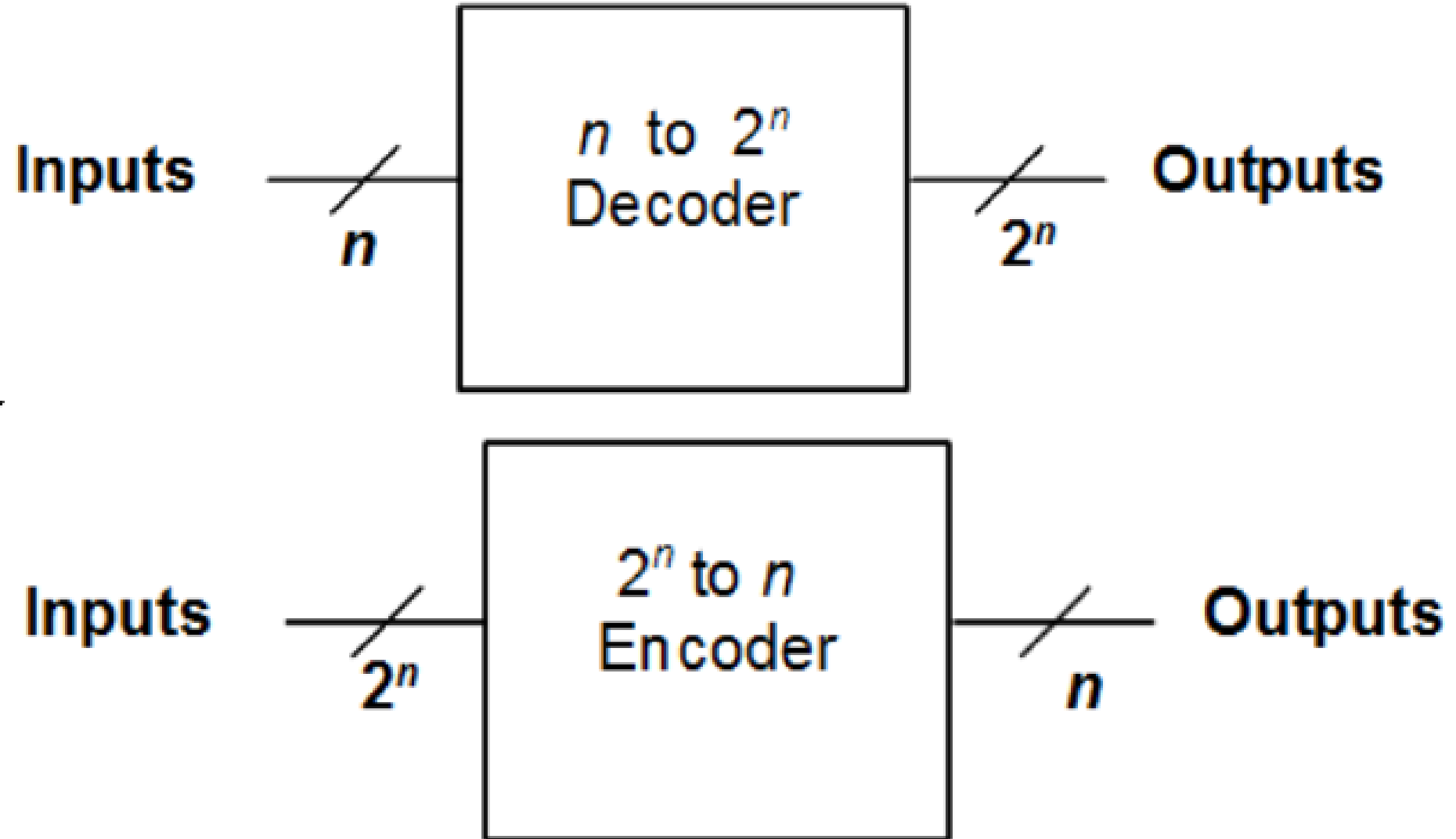
Decoder, MUX and DMUX

Digital circuits use data selection and distribution methods.

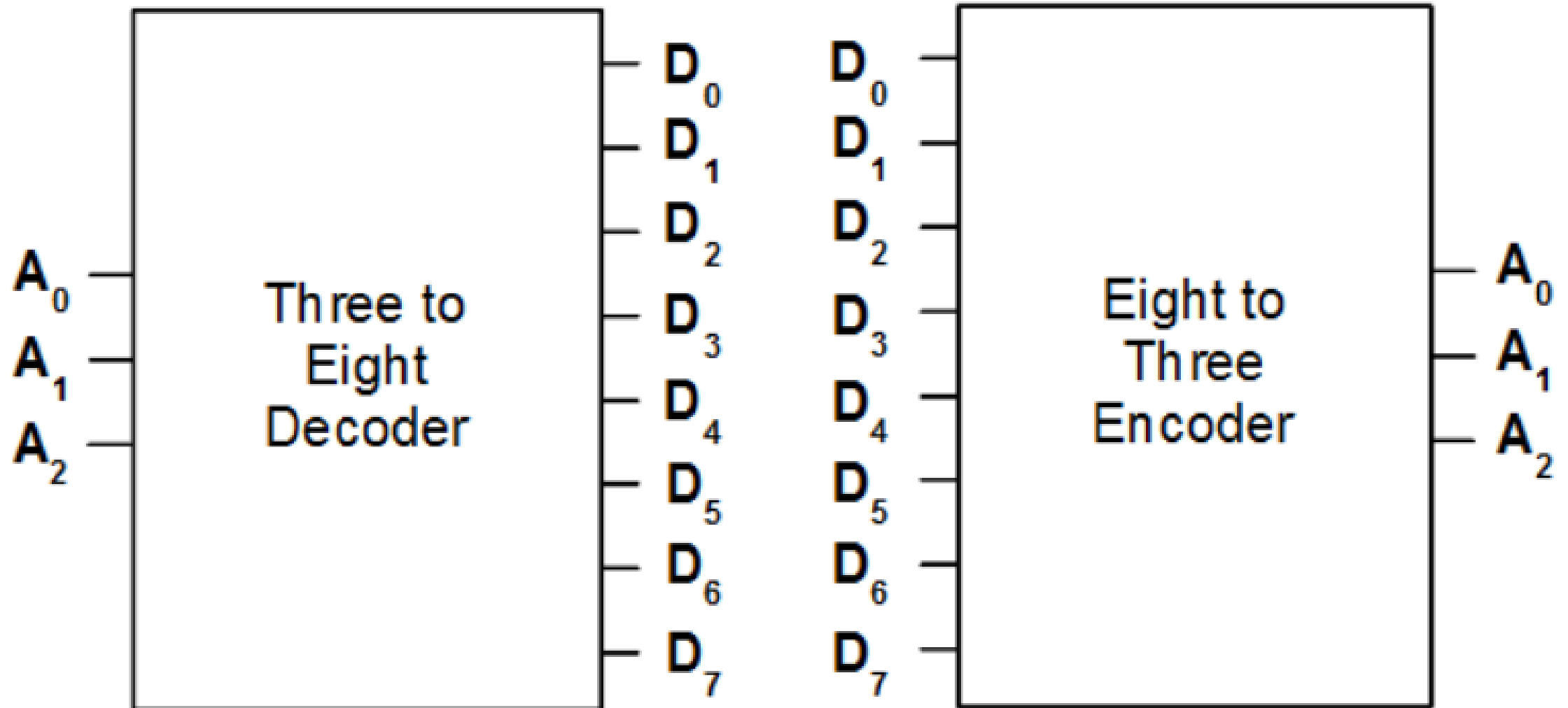
Three essential components:

- 1. Decoder,**
- 2. Multiplexer (MUX), and**
- 3. Demultiplexer (DEMUX).**

Inputs and Outputs of Decoder and Encoder



Decoder 3x8 and Encoder 8x3



Decoder

A_2	A_1	A_0	D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	1	0	0	0	0	0	0	1	0	0
0	1	1	0	0	0	0	1	0	0	0
1	0	0	0	0	0	1	0	0	0	0
1	0	1	0	0	1	0	0	0	0	0
1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	0	0	0	0	0	0	0

74138: 3x8 line decoder

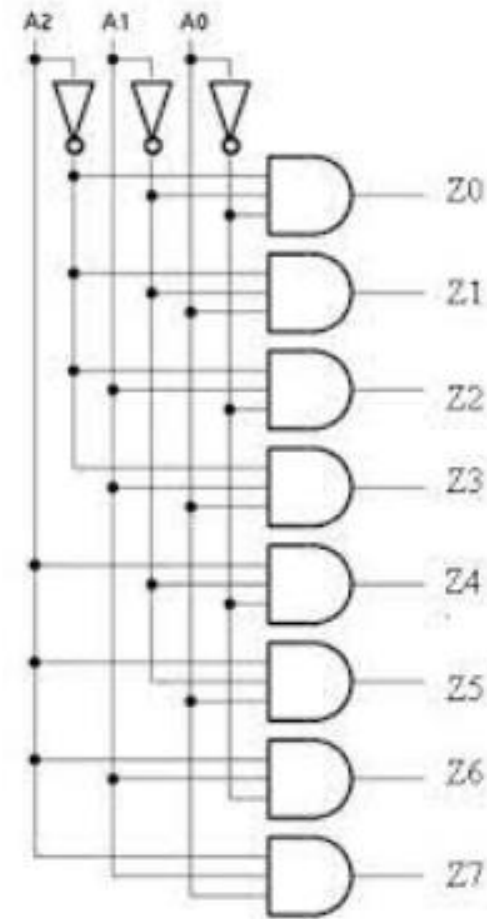
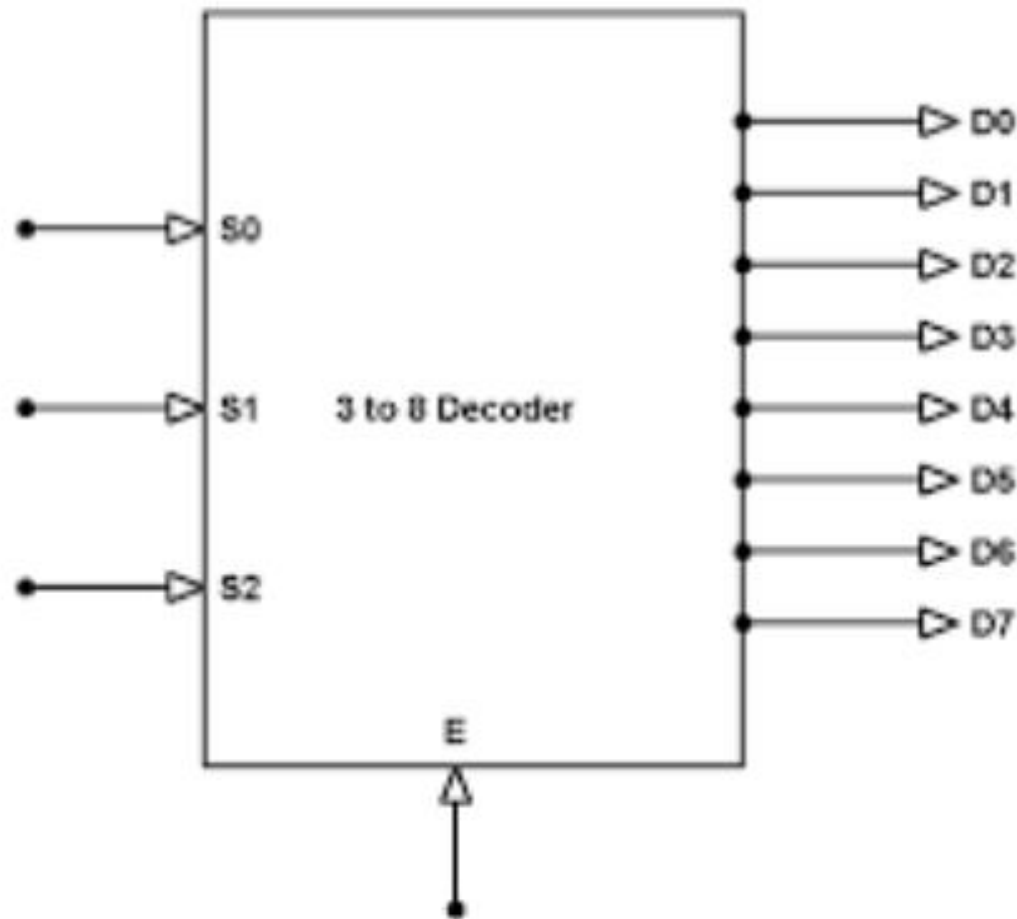
The **74138 (74LS138)** is a **3-to-8 line decoder/demultiplexer**. It takes a **3-bit binary input** and activates one of its **8 outputs** (D0 to 7) while keeping the others LOW. This IC is commonly used in memory addressing, demultiplexing, and other digital logic applications.

The decoder works only when:

- **G1 = 1** (HIGH)
- **G2A = 0** (LOW)
- **G2B = 0** (LOW)

74138

3 to 8 Line Decoder



Features of 74138

- It is very fast and high speed IC.
- It consumes very low power because it consists of low power Schottky diodes.
- It has a demultiplexing facility.
- It has a very short propagation delay.
- Appropriate operating temperature is from 0°C to 70°C.

Decoder with Active Low output

Implementing Logic Expression Using Decoder

1. SOP (Sum of Products) Implementation

Given:

$$F = \Sigma m(1, 3, 5, 7)$$

Expanded Form:

$$F = \overline{A}\overline{B}C + \overline{A}BC + A\overline{B}C + ABC$$

Using 3×8 Decoder:

- Inputs: A, B, C
- Outputs: D_0 to D_7
- Select outputs: D_1, D_3, D_5, D_7

Final Implementation:

$$F = D_1 + D_3 + D_5 + D_7$$

Decoder with Active Low output

2. POS (Product of Sums) Implementation

Given:

$$F = \Pi M(1, 3, 5, 7)$$

Expanded Form:

$$F = (A + B + \overline{C})(A + \overline{B} + C)(\overline{A} + B + \overline{C})(\overline{A} + \overline{B} + C)$$

Using Decoder:

- Select outputs corresponding to maxterms: D_1, D_3, D_5, D_7

Final Implementation:

$$F = D_1 \cdot D_3 \cdot D_5 \cdot D_7$$

Decoder with Active High output

1. SOP using Active HIGH Decoder

Given:

$$F = \Sigma m(1, 3, 5, 7)$$

- Decoder outputs $D_1, D_3, D_5, D_7 = 1$
- To “add” them → use OR gate

Final:

$$F = D_1 + D_3 + D_5 + D_7$$

Use OR gate for SOP (Active HIGH outputs)

Decoder with Active High output

2. POS using Active HIGH Decoder

Given:

$$F = \Pi M(1, 3, 5, 7)$$

$$F = (A + B + C)(A + \bar{B} + C)(\bar{A} + B + C)(\bar{A} + \bar{B} + C)$$

So the implementation is

Take outputs D1, D3, D5, D7 from the decoder and connect them directly to a NOR gate. The output of the NOR gate gives F.

Final result

$$F = \Pi M(1, 3, 5, 7)$$

$$F = (D1 + D3 + D5 + D7)'$$

$$F = \text{NOR}(D1, D3, D5, D7)$$

BCD to Decimal Decoder

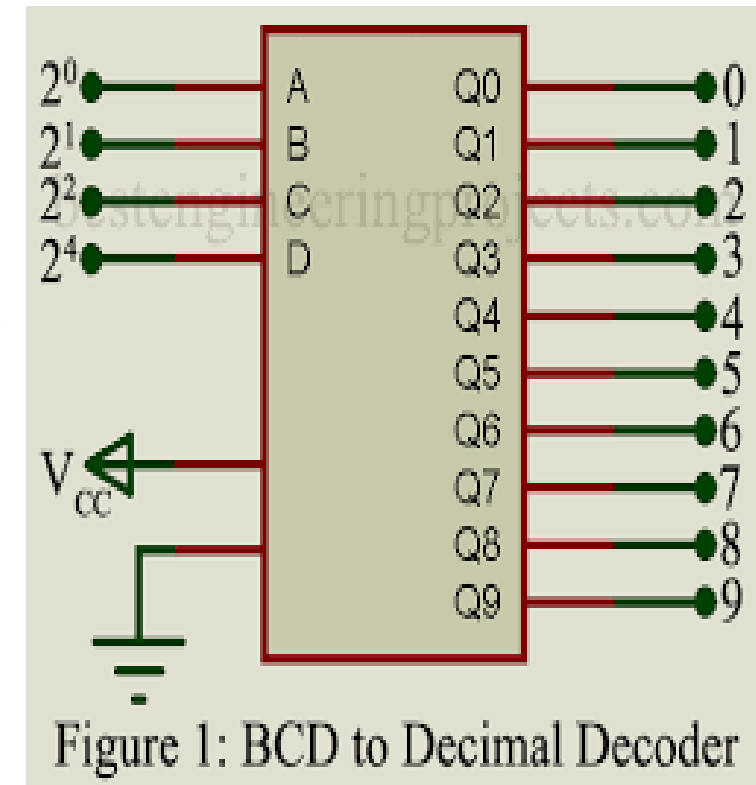
Truth Table - BCD to Decimal Decoder

A	B	C	D	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	D_8	D_9
0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0
0	1	1	0	0	0	0	0	0	0	1	0	0	0
0	1	1	1	0	0	0	0	0	0	0	1	0	0
1	0	0	0	0	0	0	0	0	0	0	0	1	0
1	0	0	1	0	0	0	0	0	0	0	0	1	0

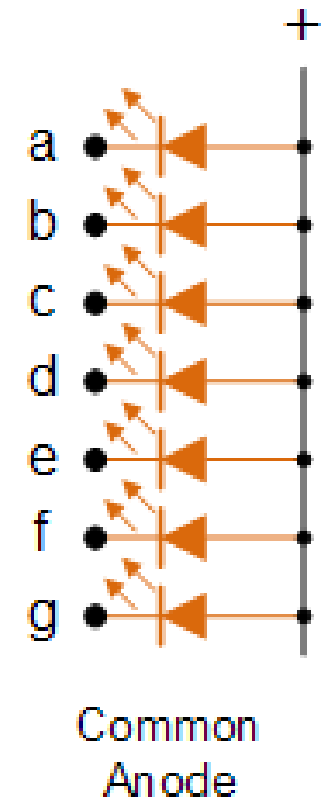
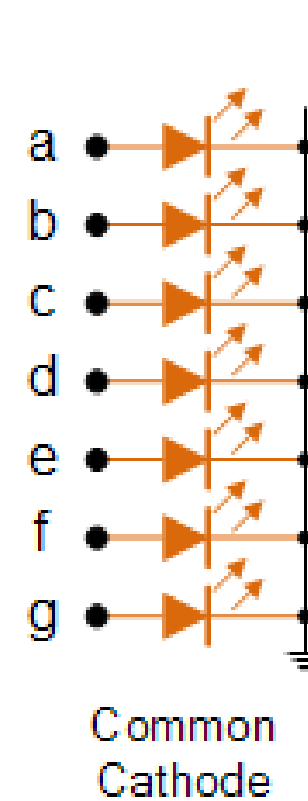
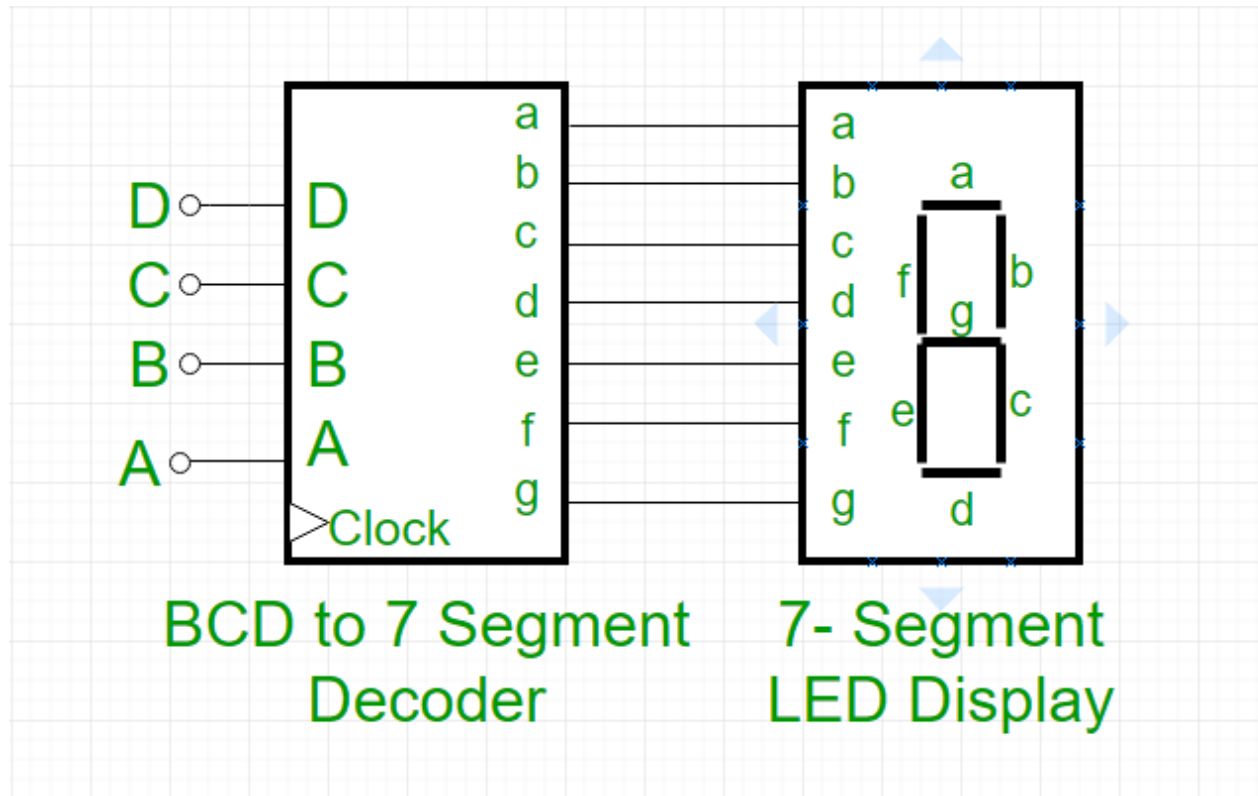
Don't Care Terms

10, 11, 12, 13, 14, 15

AB \ CD	00	01	11	10
00		D_1		
01				
11	X	X	X	X
10			X	X

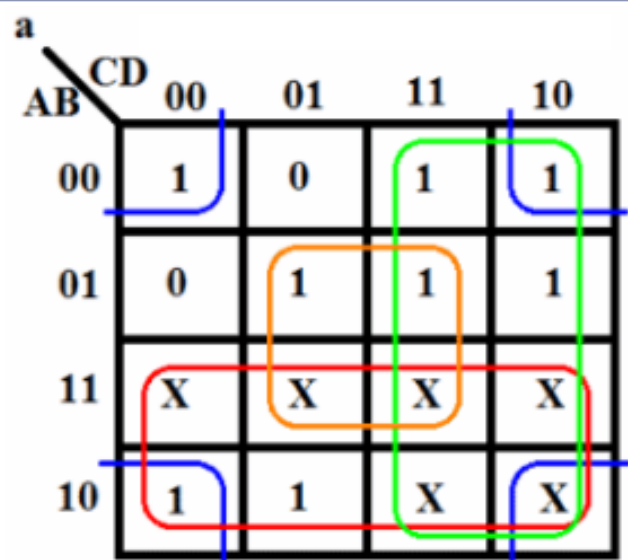


BCD to 7 Segment Decoder

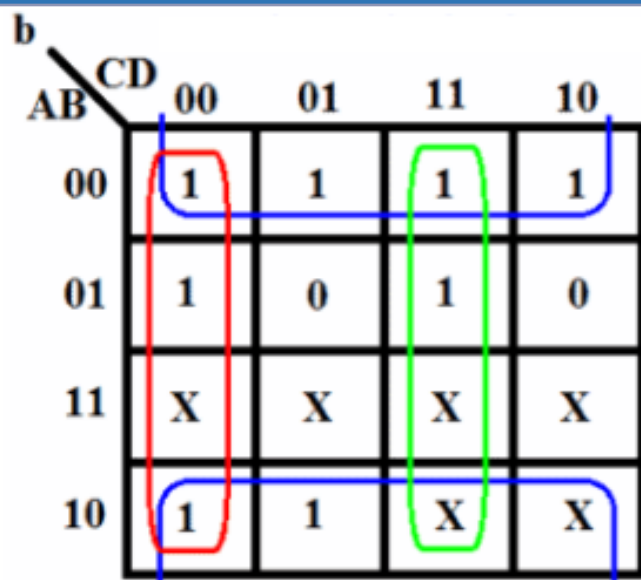


Truth table for common cathode type BCD to seven segment decoder

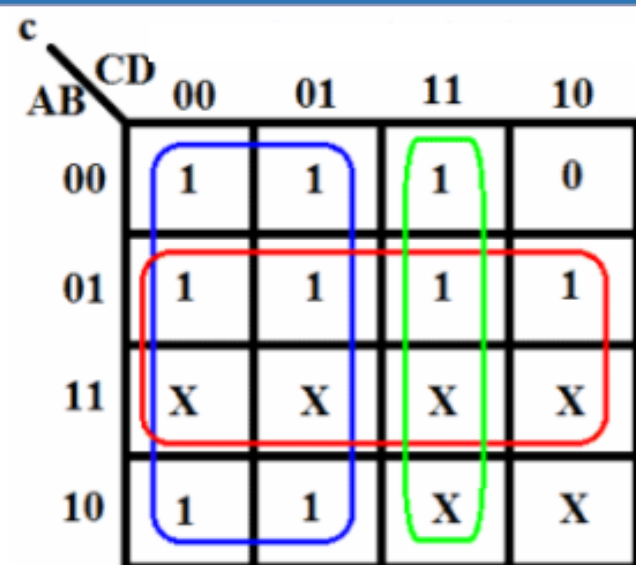
Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	1	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	1	0	1	1	



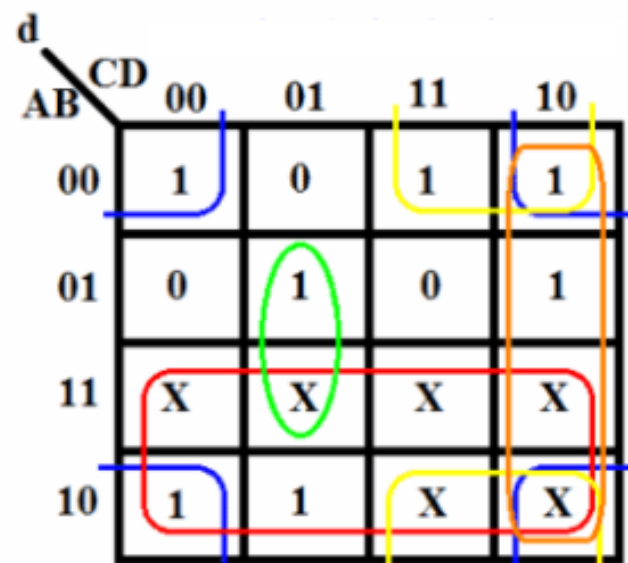
$$a = A + C + BD + \bar{B}D$$



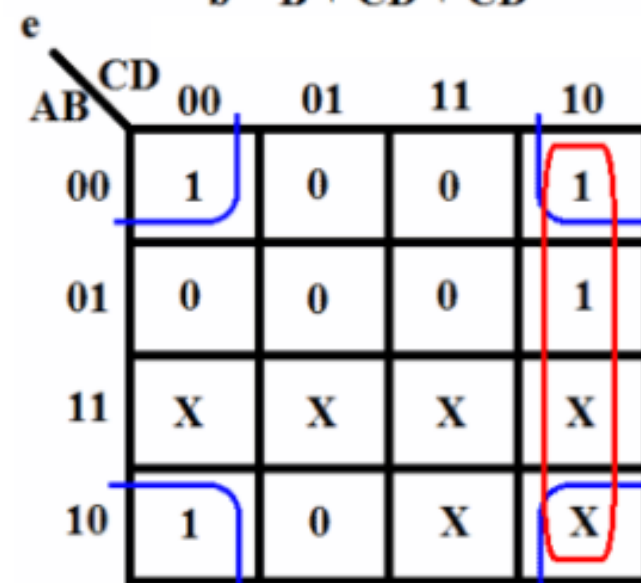
$$b = \bar{B} + \bar{C}\bar{D} + CD$$



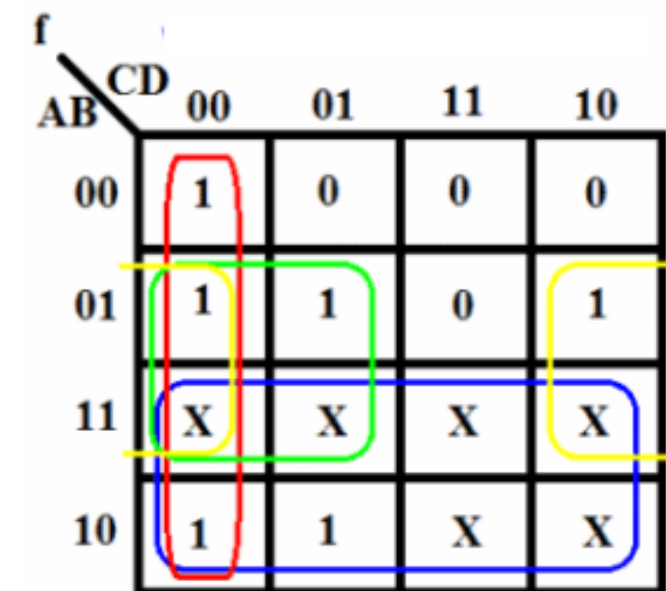
$$c = B + \bar{C} + D$$



$$d = A + \bar{B}\bar{D} + \bar{B}C + \bar{C}\bar{D} + B\bar{C}\bar{D}$$



$$e = \bar{B}\bar{D} + \bar{C}\bar{D}$$



$$f = A + B\bar{C} + \bar{B}\bar{D} + \bar{C}\bar{D}$$

g

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

$$g = A + B\bar{C} + \bar{B}C + C\bar{D}$$

MUX

1. Time Division Multiplexing (TDM)

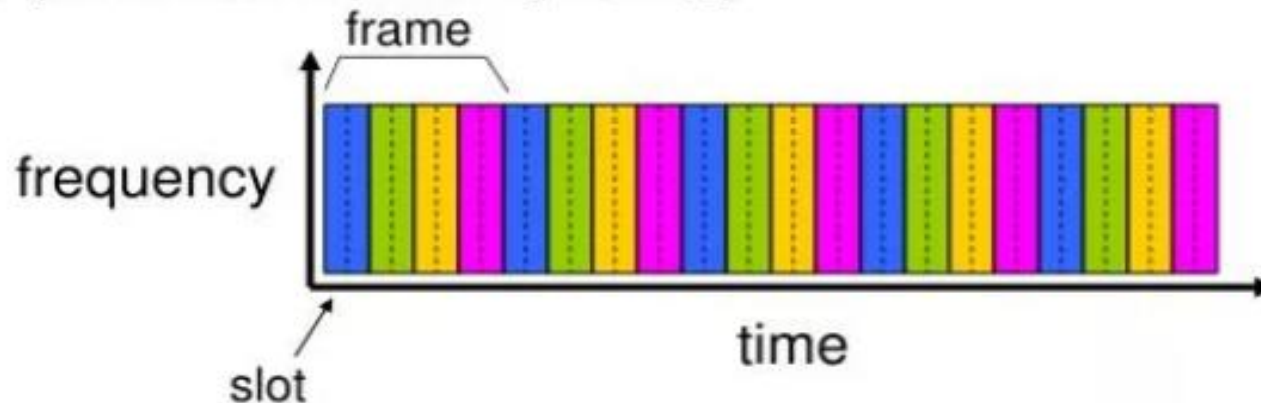
Concept: Each signal is assigned a specific time slot in a cyclic manner.

Types:

- **Synchronous TDM:** Fixed time slots for each signal, even if no data is present.
- **Asynchronous (Statistical) TDM:** Time slots are dynamically allocated based on data availability.

Example: Digital telephony, where multiple phone calls share the same transmission line.

TDM (Time division multiplexing)



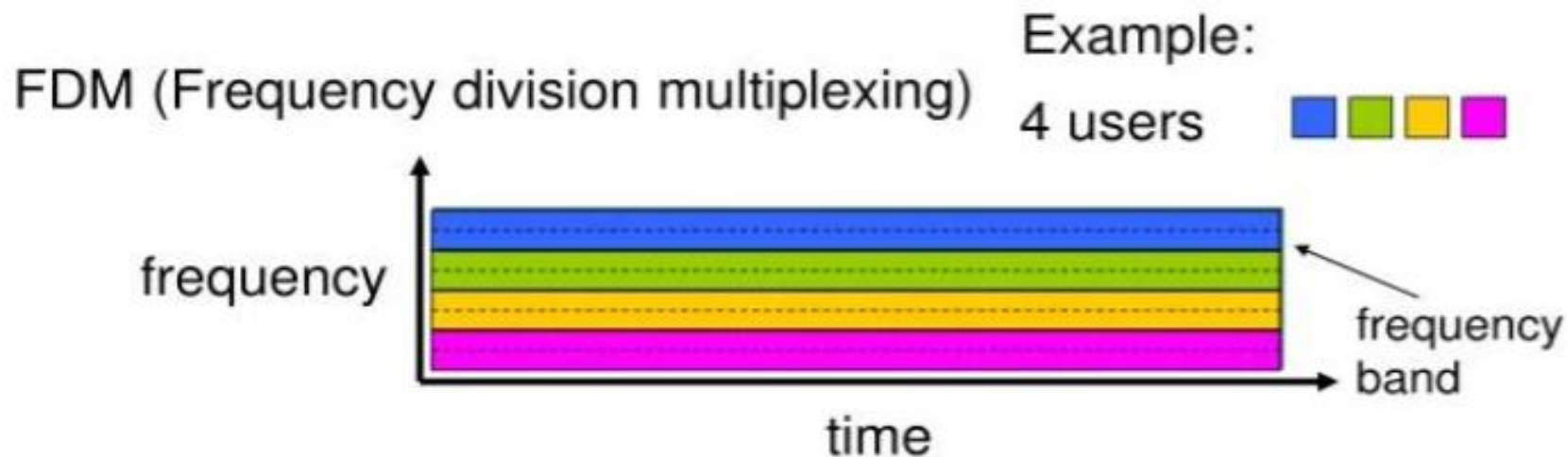
MUX

2. Frequency Division Multiplexing (FDM)

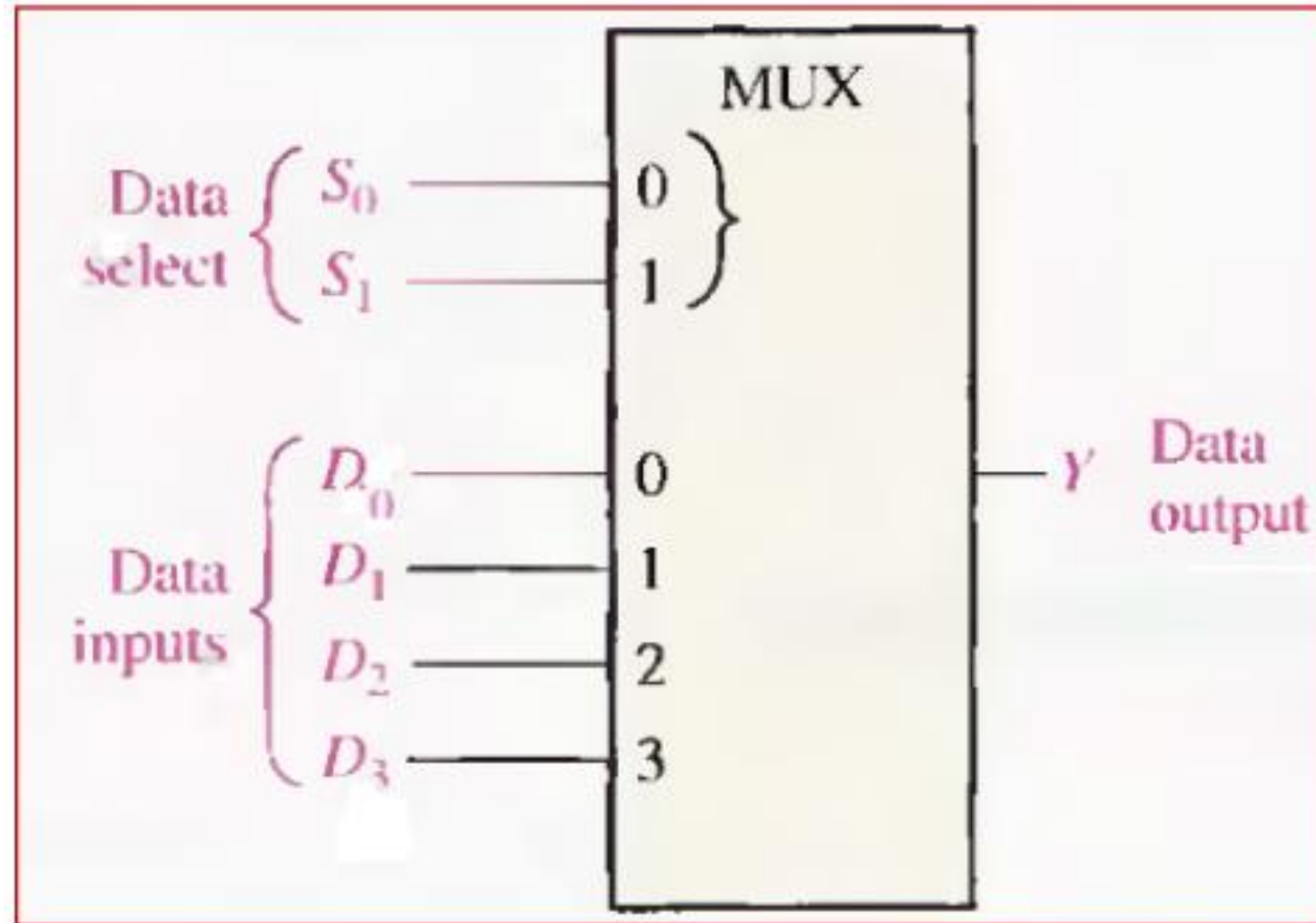
Concept: Each signal is assigned a unique frequency band within the available spectrum.

Working: Signals are modulated at different carrier frequencies and transmitted simultaneously.

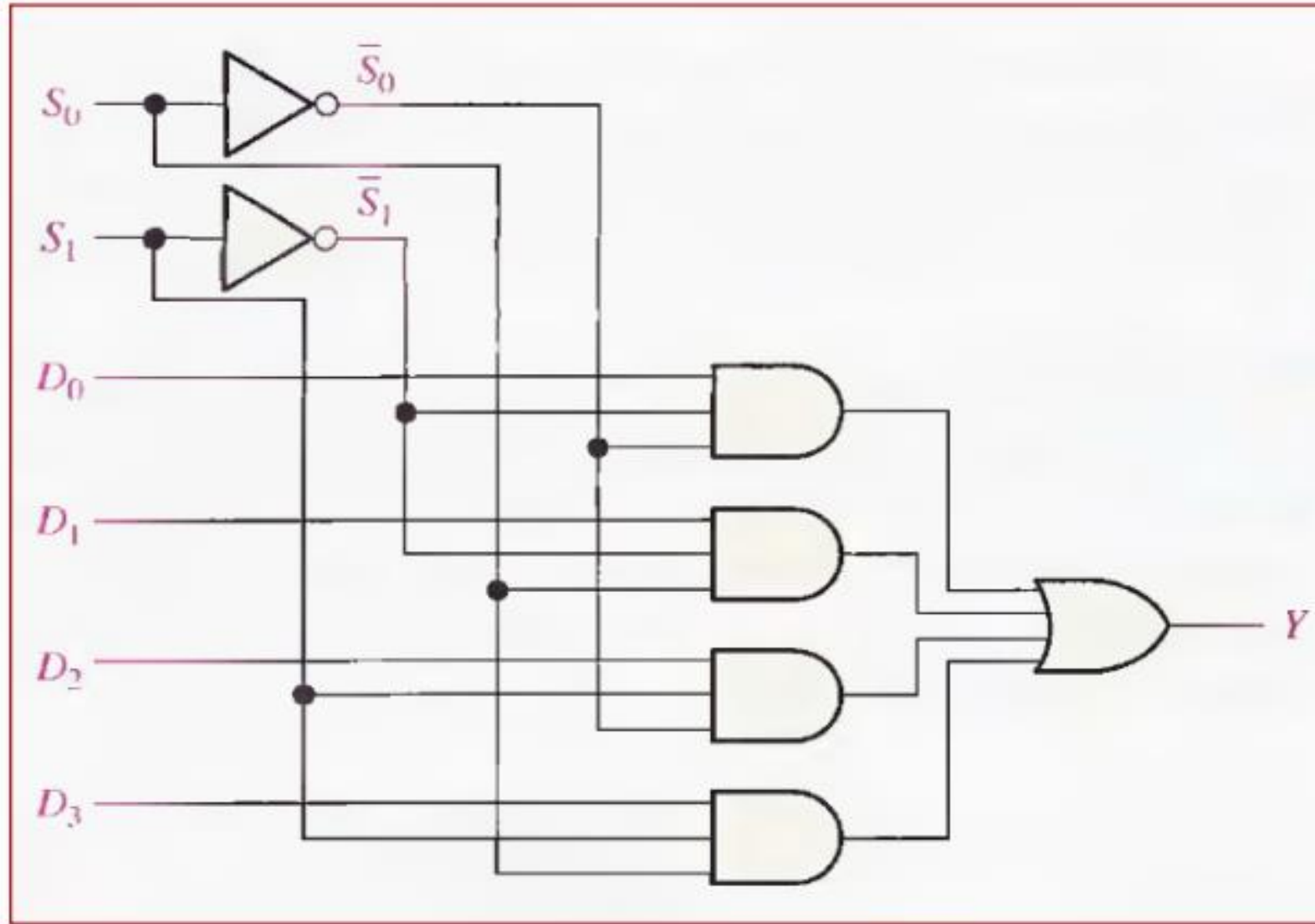
Example: FM/AM radio broadcasting, where different stations operate at different frequencies.



4 input MUX



4 input MUX



4 input MUX

There are two data-select lines because with two select bits. Anyone of the four data-input lines can be selected.

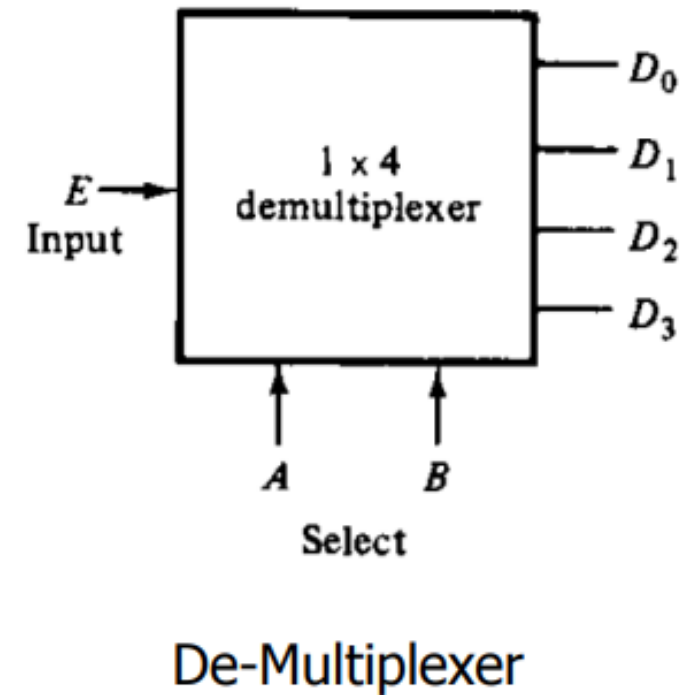
DATA-SELECT INPUTS		INPUT SELECTED
S_1	S_0	
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

$$Y = D_0 S'_1 S'_0 + D_1 S'_1 S_0 + D_2 S_1 S'_0 + D_3 S_1 S_0$$

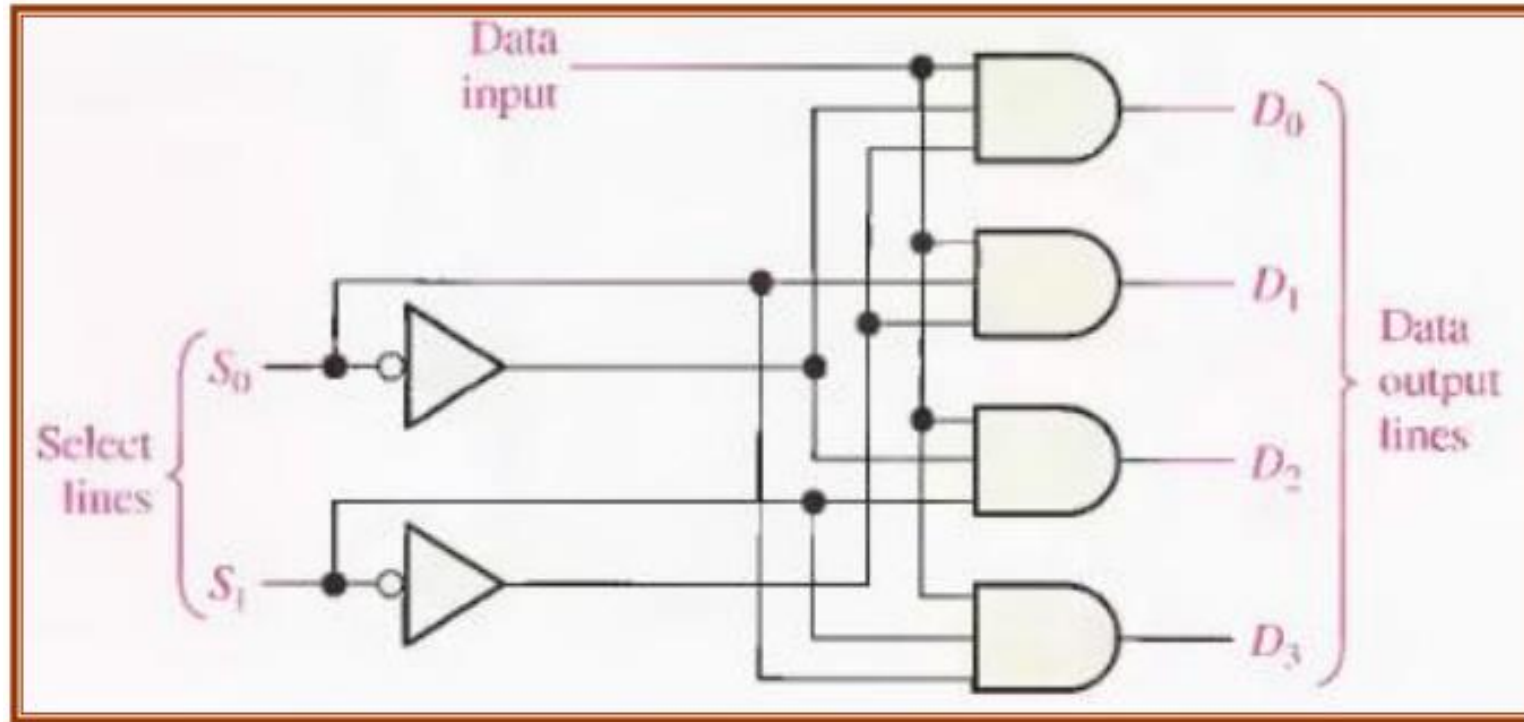
DMUX

Basic functionalities

- A Demultiplexer (DEMUX) basically reverses the multiplexing function.
- It takes digital information from one line and distributes it to a given number of output lines.
- For this reason, the demultiplexer is also known as a data distributor.
- Decoders can also be used as demultiplexers.



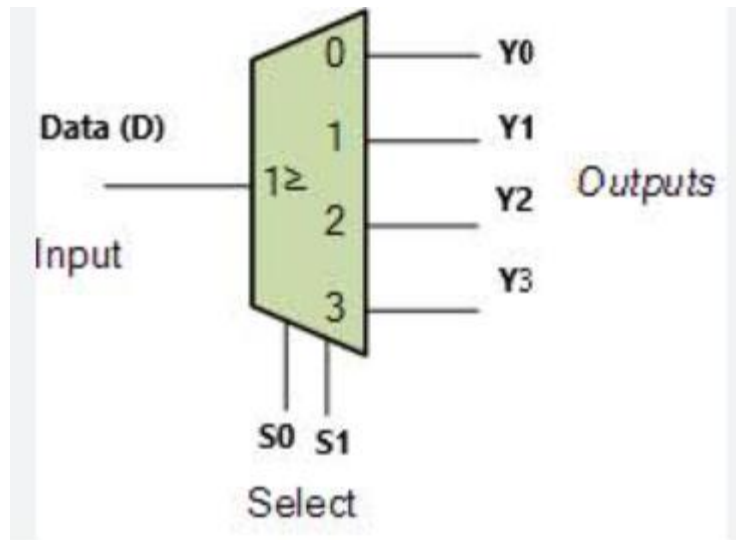
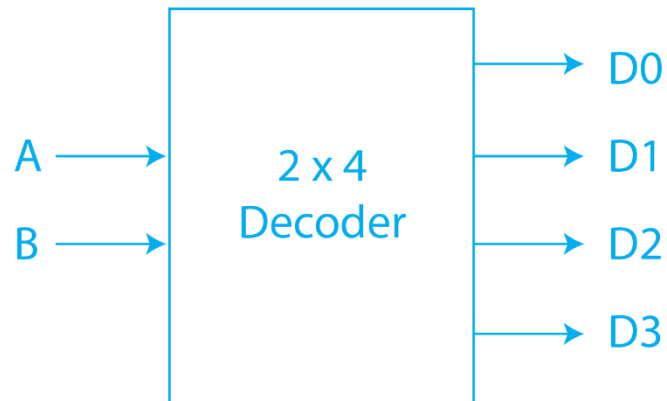
1x4 DMUX



E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

Decoder as DMUX

A decoder works as a demultiplexer (DEMUX) by using its Enable (E) pin as the data input and its two address inputs (A_1 , A_0) as the select lines (S_1 , S_0). The input data is routed to one of the four outputs (Y_0 - Y_3) based on the select lines, provided the decoder is enabled.



MUX implementation

If $F(A,B,C) = (1,2,4,5)$

If A and B is selected as Selected input, then

<i>A</i>	<i>B</i>	<i>C</i>	<i>F</i>
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$F = C$

$F = C'$

$F = 1$

$F = 0$

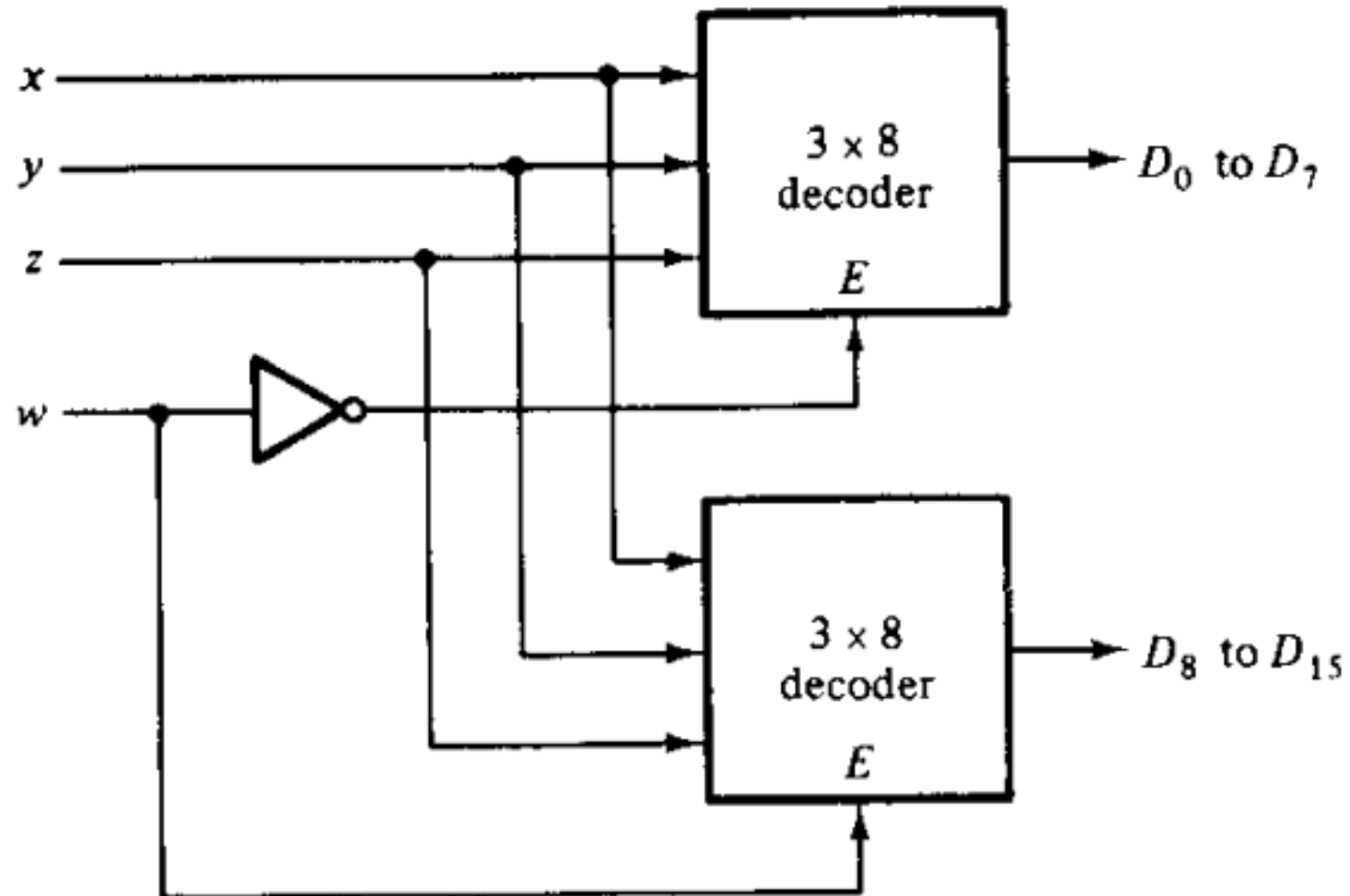
MUX implementation

Implement the following function with a multiplexer:

$$F(A,B,C,D)=m(1,4,5,7,9,12,13)$$

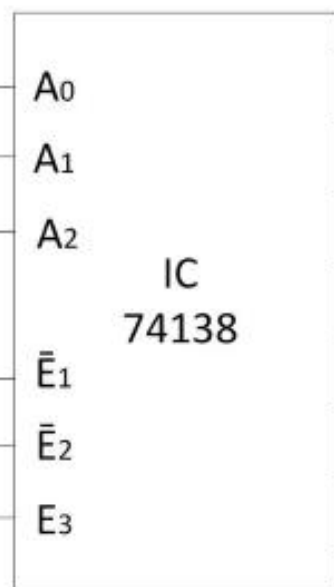
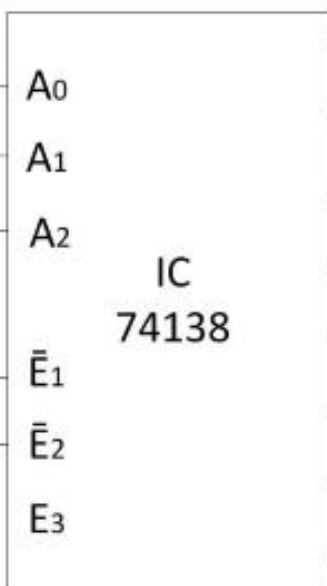
CASCADING

Q: Cascade two 3x8 line Decoder to form a 4x16 Decoder



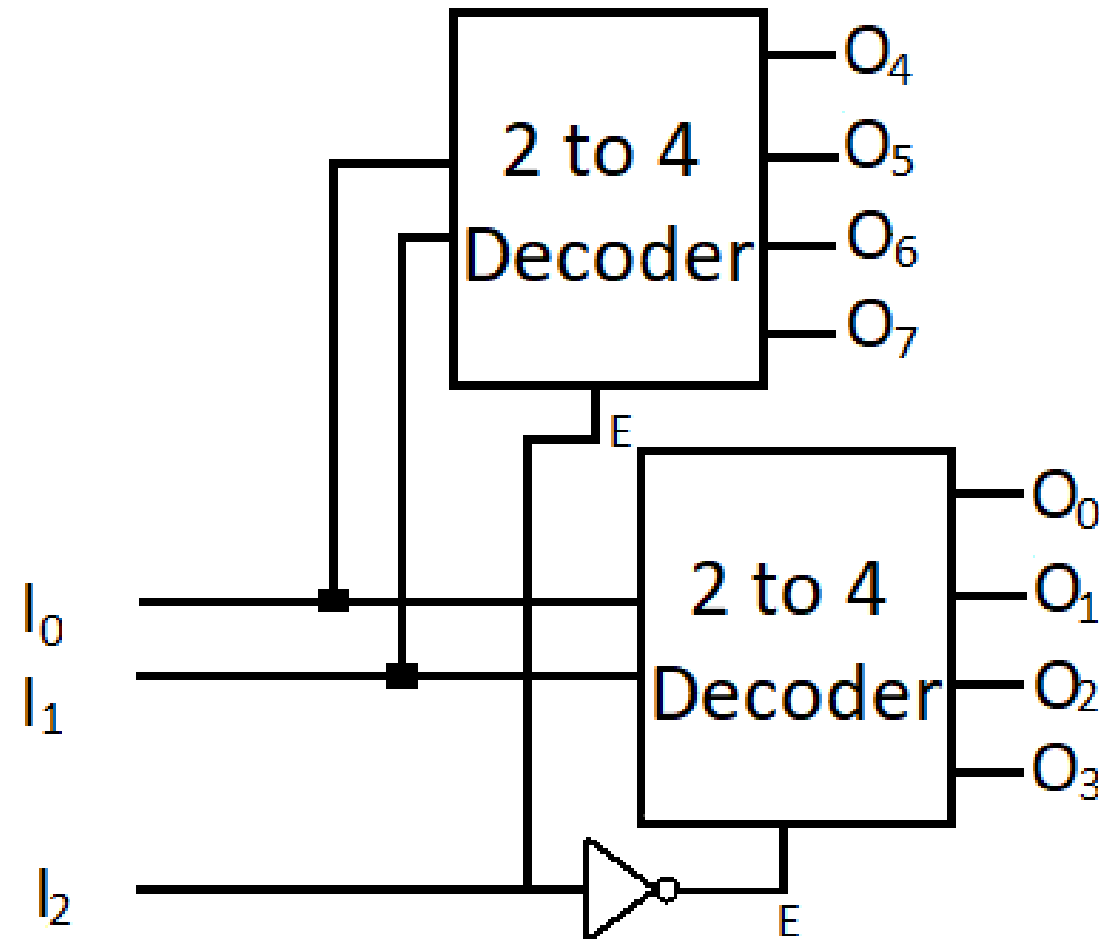
BINARY
INPUTS

A₀
A₁
A₂
A₃

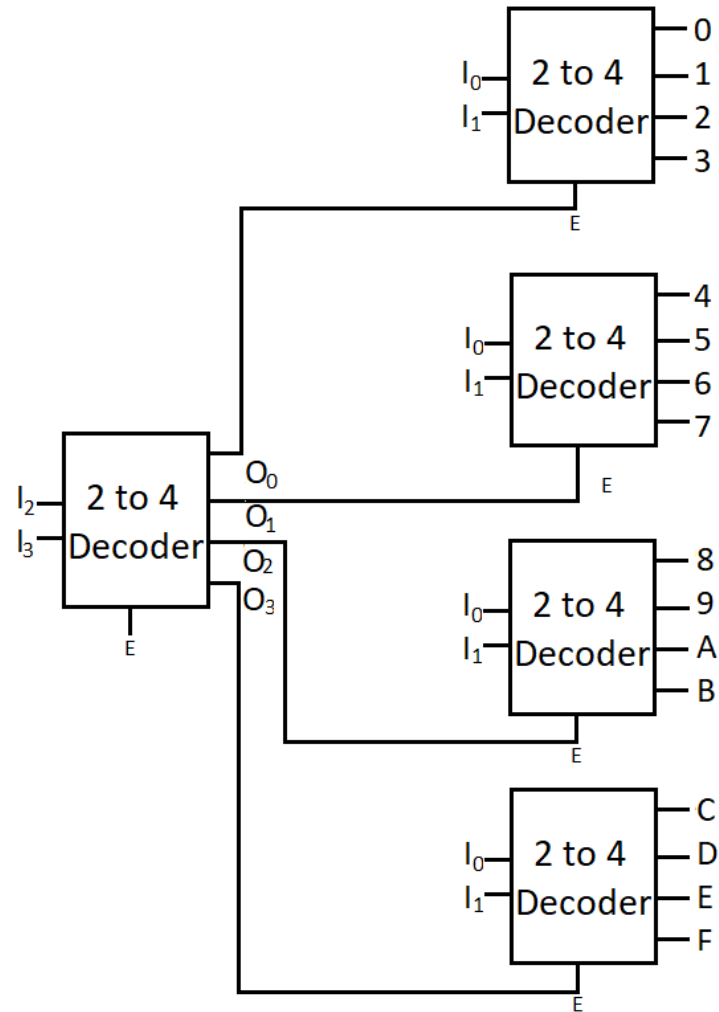


DECIMAL
OUTPUTS

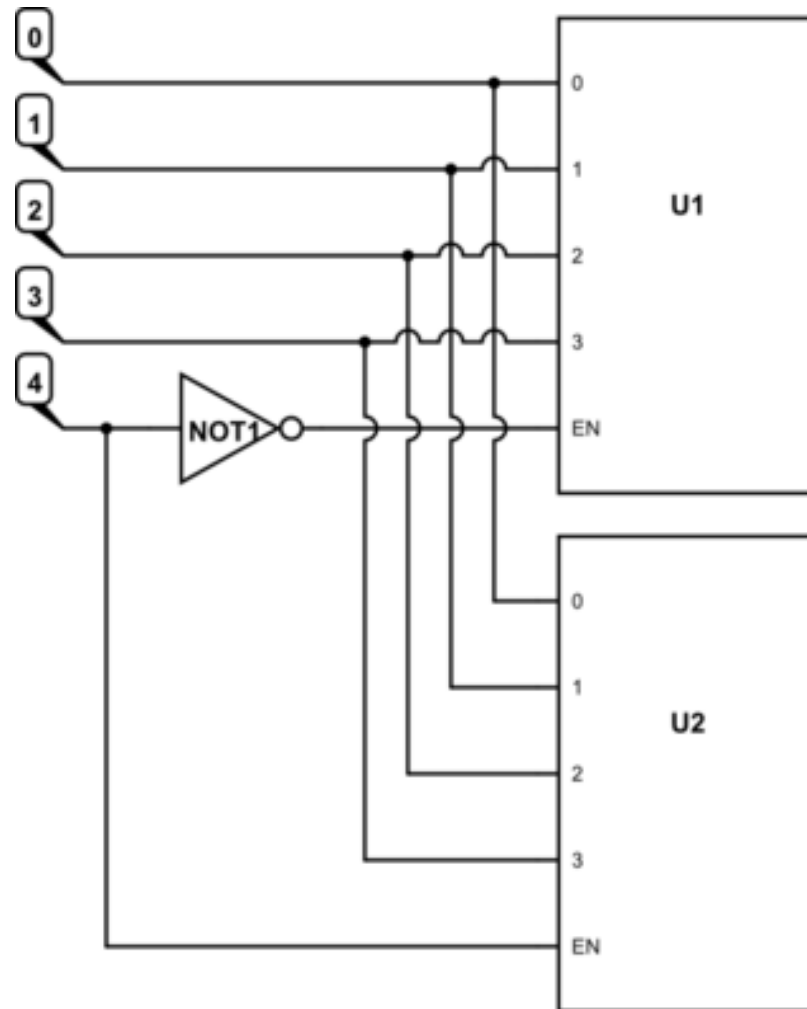
Q: Cascade two 2x4 line Decoder to form a 3x8 Decoder



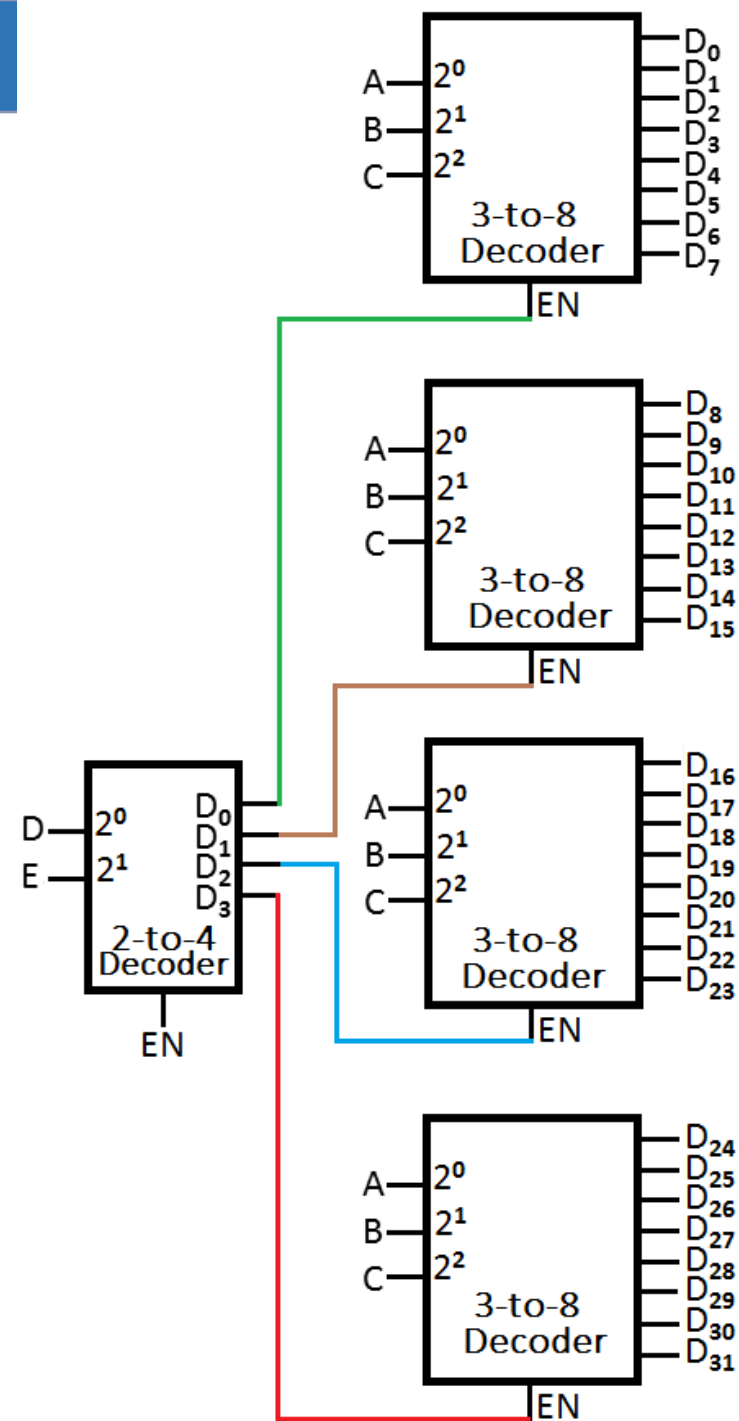
Q: Cascade two 2x4 line Decoder to form a 4x16 Decoder



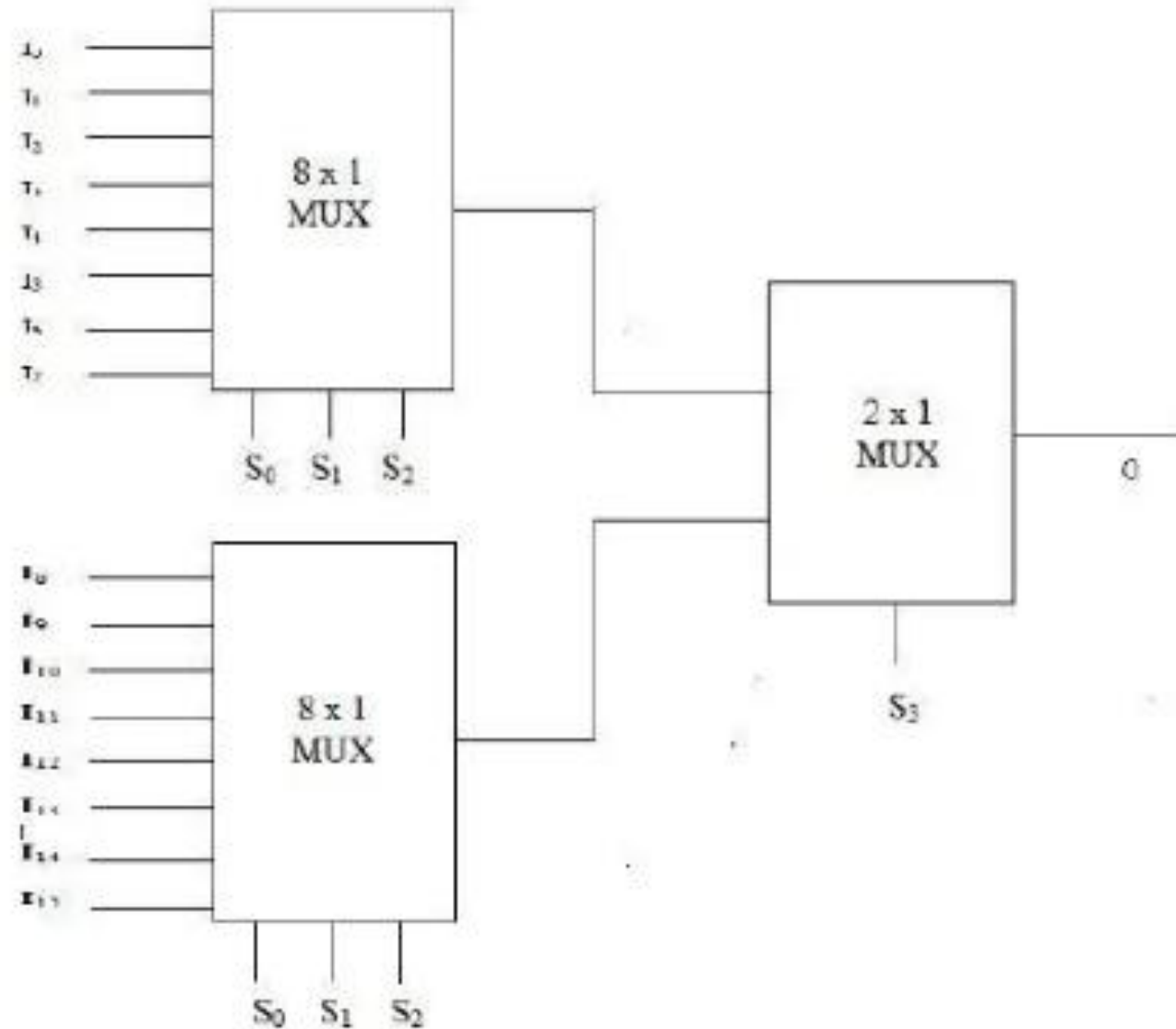
Q: Cascade two 4x16 line Decoder to form a 5x32 Decoder



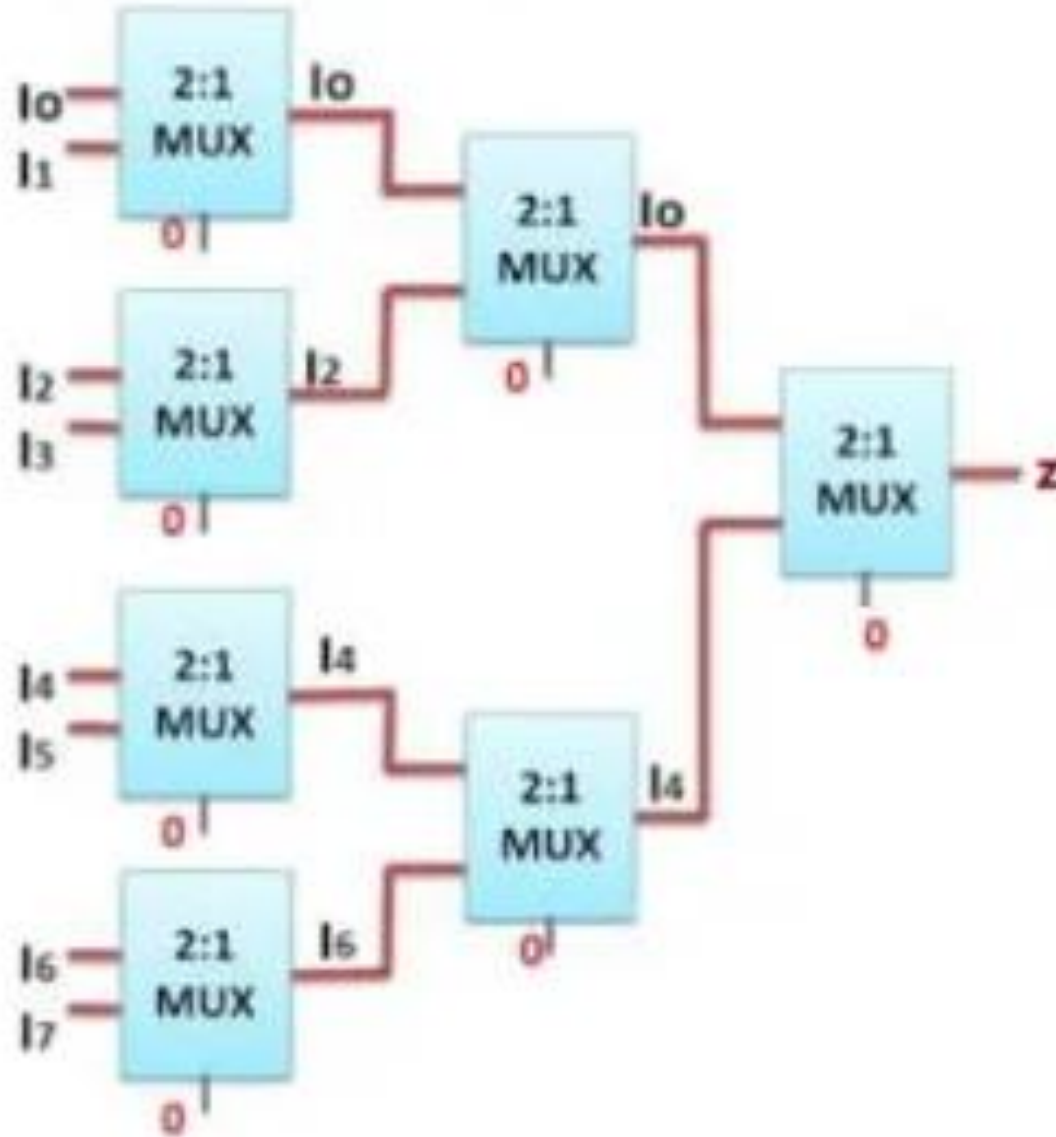
Q: Cascade four 3x8
line Decoder to form a
5x32 Decoder



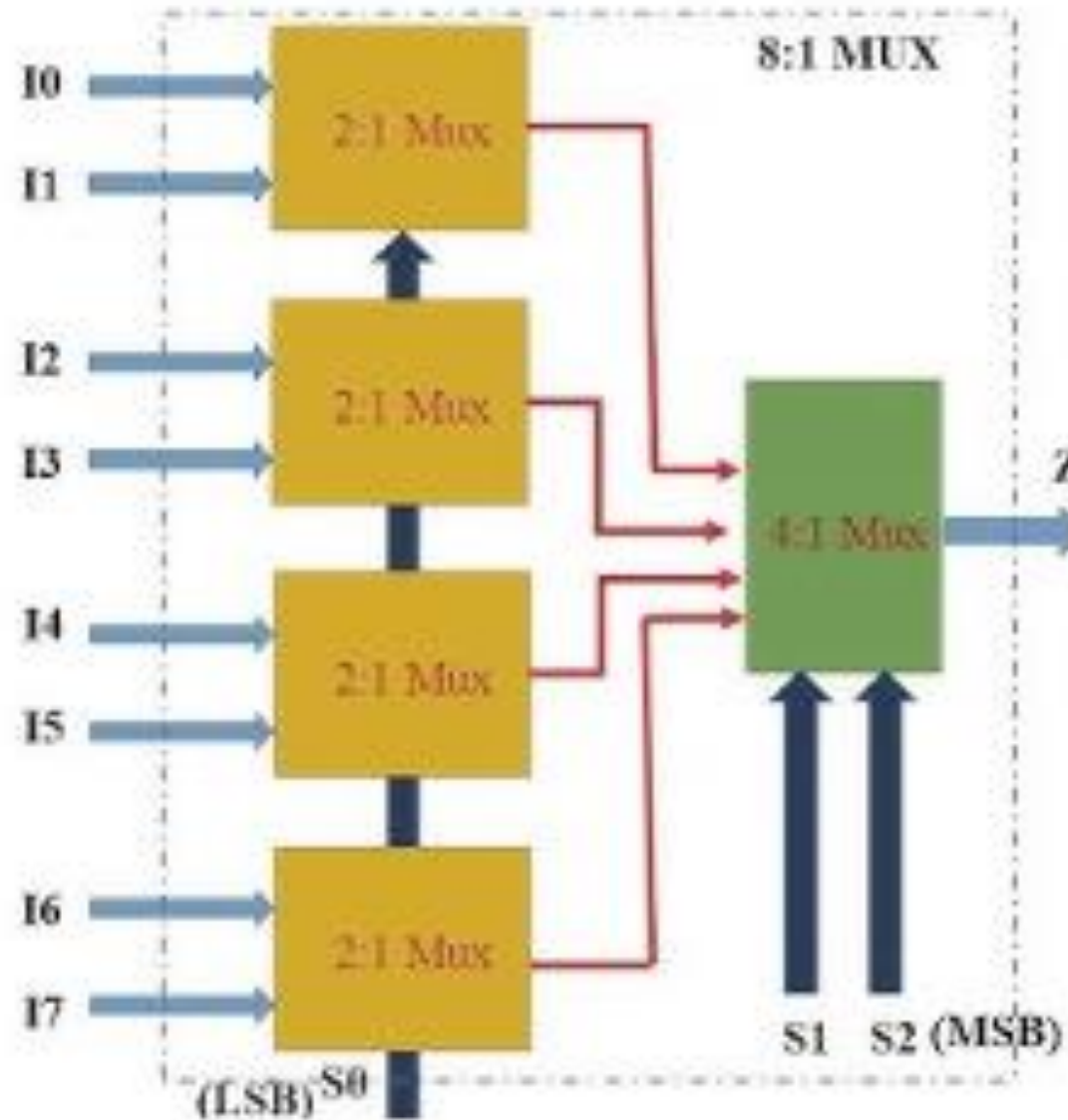
Q: Cascade four 8x1 MUX to form a 16x1 MUX



Q: Cascade 16x1 MUX using only 2x1 MUX

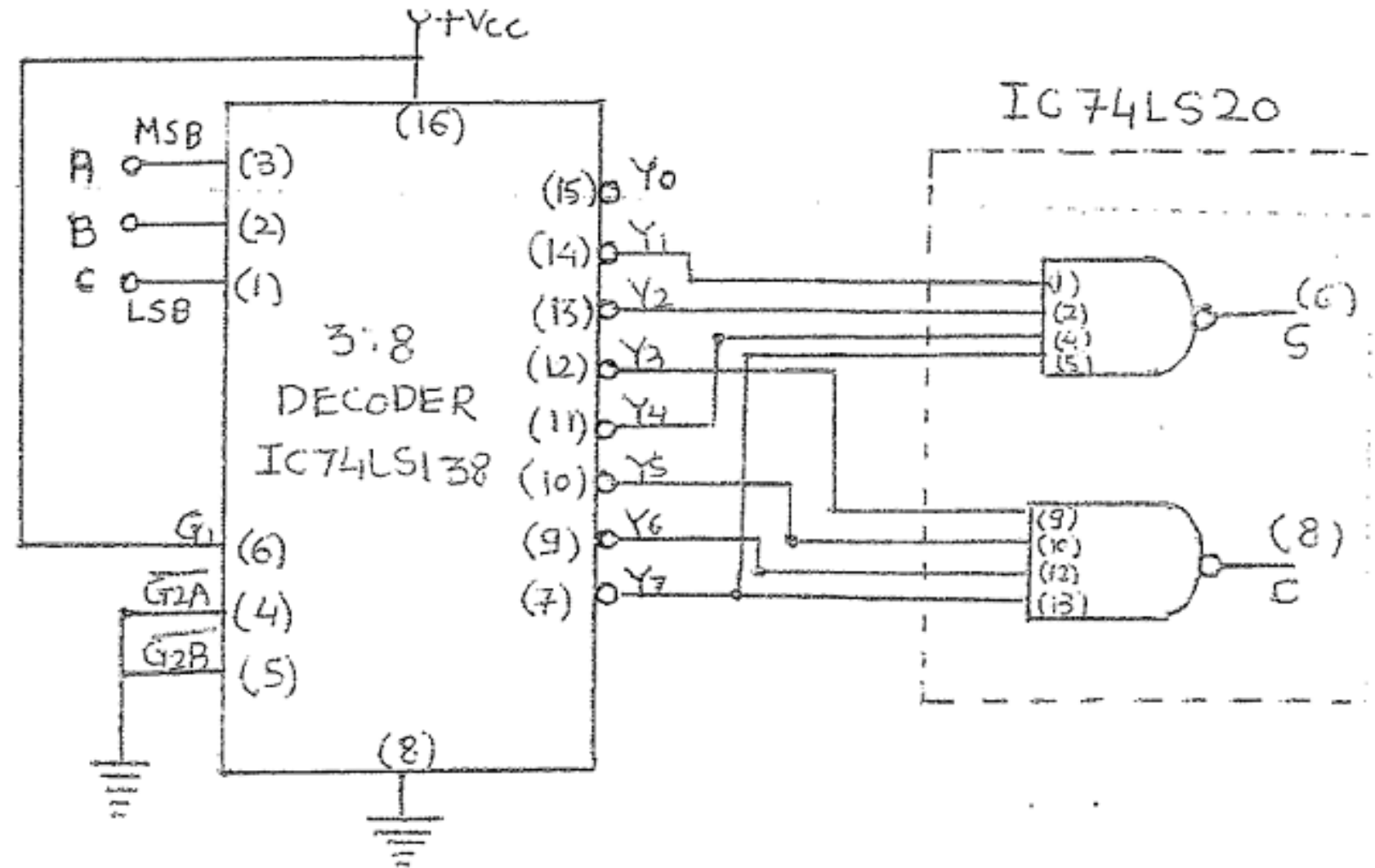


Q: Cascade 16x1 MUX using 2x1 MUX

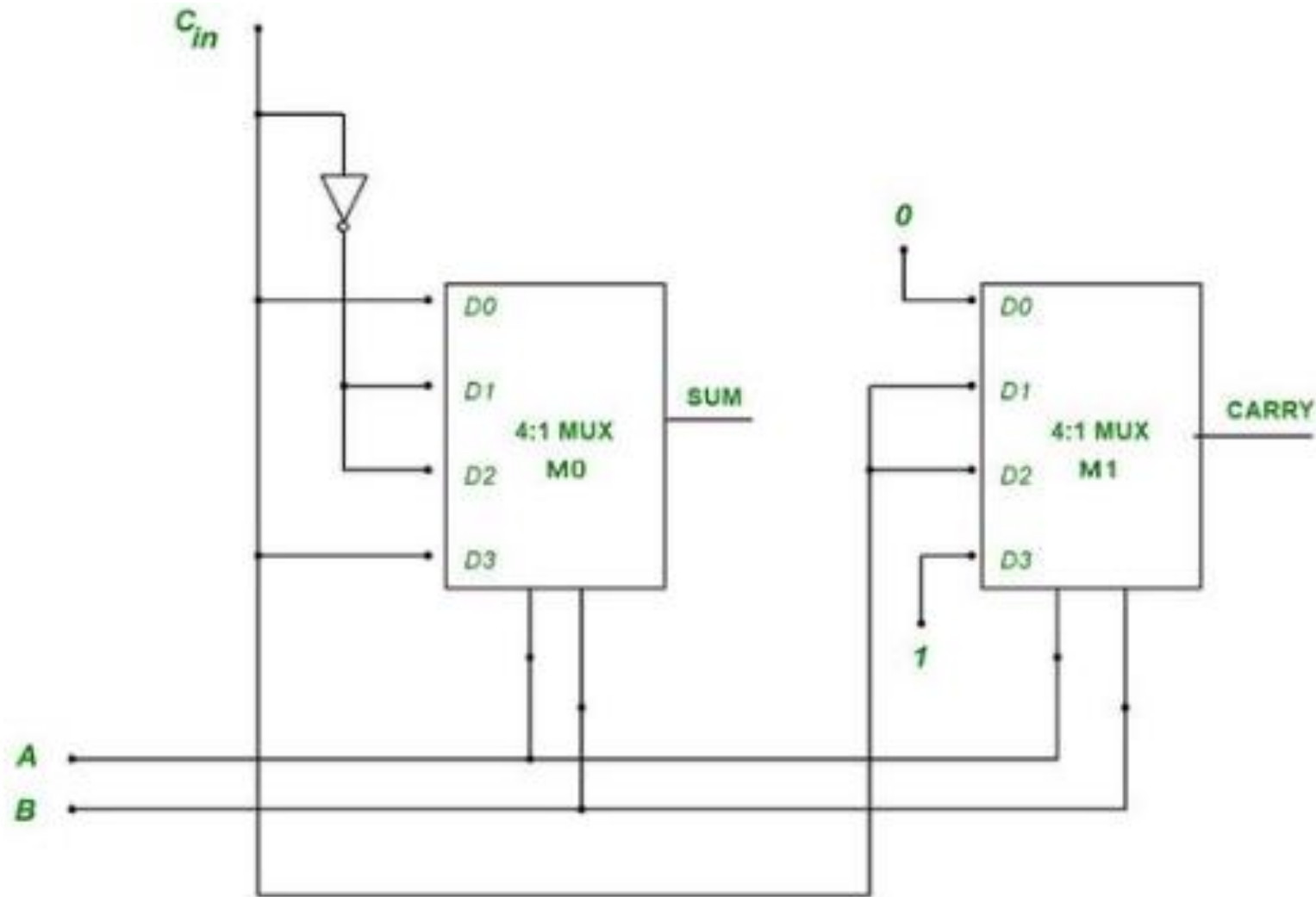


Design full adder using 3x8 Decoder

A	B	<u>Cin</u>	S	<u>Cout</u>
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

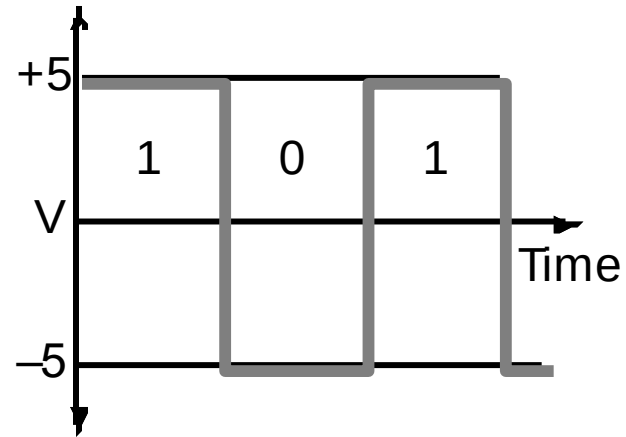


Design full adder using MUX

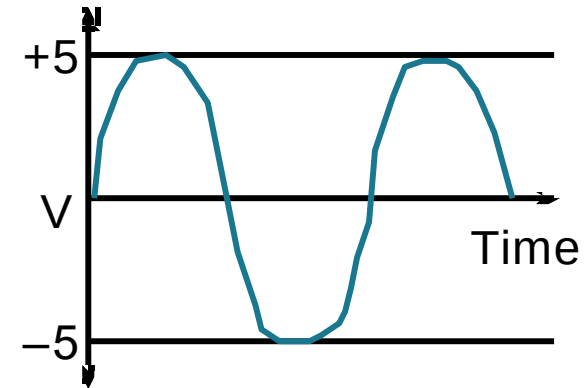


REVISION

Digital vs. Analog



Digital:
only assumes discrete values



Analog:
values vary over a broad range
continuously

Number Systems

A number system is a way of writing and representing numbers using symbols according to consistent rules.

Non-positional Number Systems

Characteristics

- Use symbols such as I for 1, II for 2, III for 3, IIII for 4, IIIII for 5, etc.
- Each symbol represents the same value regardless of its position in the number.
- The symbols are simply added to find the value of a particular number.

Difficulty

- It is difficult to perform arithmetic with such a number system.

Number Systems

Positional Number Systems

Characteristics

- Use only few symbols called digits.
- These symbols represent different values depending on the position they occupy in the number.
- The value of each digit depends on its position and the base (radix).

Main Positional Number Systems

- Decimal Number System (Base 10)
- Binary
- Octal
- Hexadecimal

Signed number arithmetic

For positive binary numbers simple binary representation is enough to use in arithmetic.

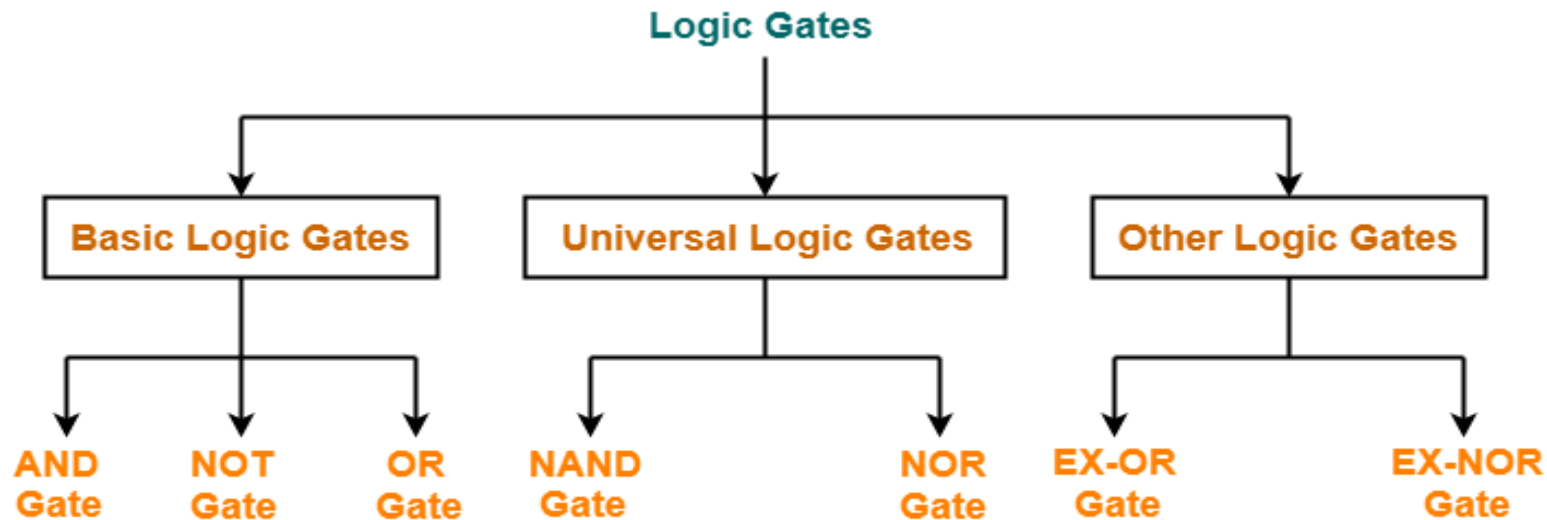
For negative binary number, 1's and 2's complement of the binary counter parts.

Key difference

Feature	1's Complement	2's Complement
Zero representation	Two (+0, -0)	One
End-around carry	Required	Not used
Overflow rule	Sign mismatch	Sign mismatch
Carry test	Not reliable	Reliable
Hardware use	Rare	Standard

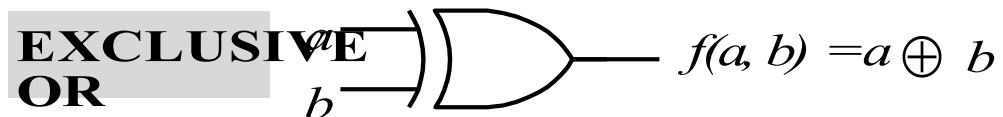
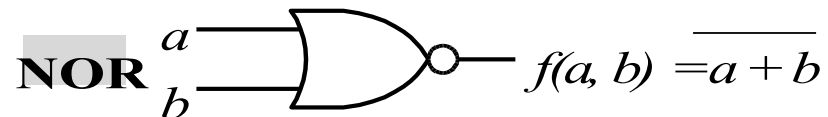
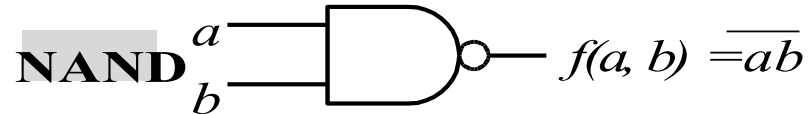
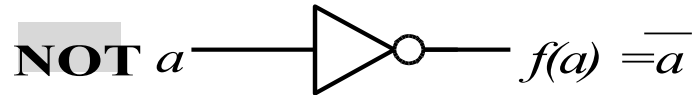
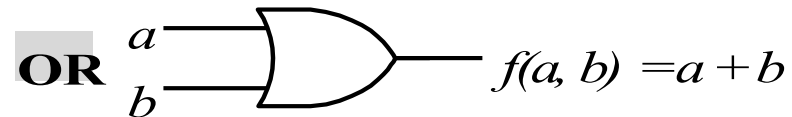
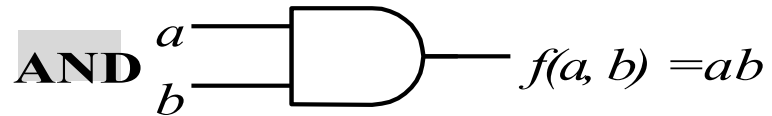
Understanding Digital Circuit Building Block

- Logic gates are basic building blocks of digital circuits. They perform logical operations on one or more binary inputs to produce a single binary output.
- Their name is derived from their ability to make decision in the sense that it produces one output when some combination of input levels are given and a different output when other combinations are present.

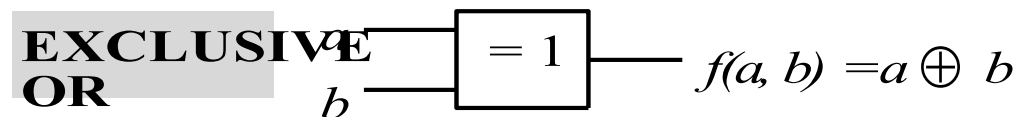
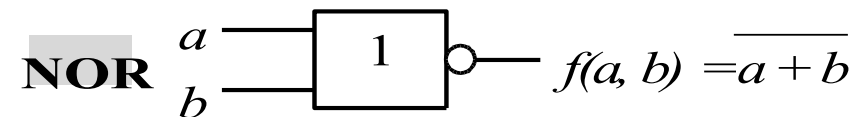
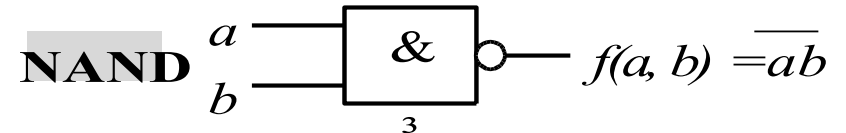
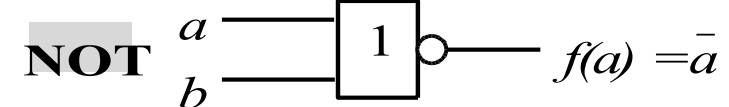
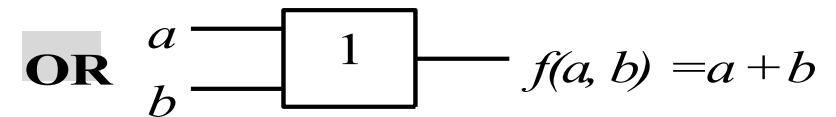
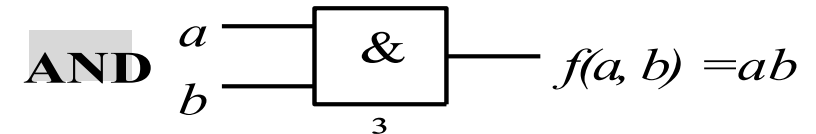


Types of Logic Gates

Electronic Logic Gates



Symbol set 1



Symbol set 2
(ANSI/IEEE Standard 91-1984)

Active High and Active Low

Active Low

Definition: Conversely, a signal is considered active low when a low voltage level (usually represented as '0' or logic low) indicates the active state of the signal.

Example: In this case, pressing a button might pull the signal to ground (0V), which is interpreted as "on" or "active." Many reset signals in digital circuits are active low, meaning they are triggered when the signal is low.

Active High: Active state = high voltage (1)

Active Low: Active state = low voltage (0)

Active High and Active Low

Positive Versus Negative Logic

	Positive Logic	Negative Logic
1 is represented by	High Voltage	Low Voltage
0 is represented by	Low Voltage	High Voltage

Basic Theorems and Laws of Boolean Algebra

1. Identity Laws:

- $A + 0 = A$
- $A \cdot 1 = A$

2. Null Laws:

- $A + 1 = 1$
- $A \cdot 0 = 0$

3. Idempotent Laws:

- $A + A = A$
- $A \cdot A = A$

4. Complement Laws:

- $A + A' = 1$
- $A \cdot A' = 0$

5. Double Negation Law:

- $(A')' = A$

6. Distributive Laws:

- $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
- $A + (B \cdot C) = (A + B) \cdot (A + C)$

7. Absorption Laws:

- $A + (A \cdot B) = A$
- $A \cdot (A + B) = A$

8. De Morgan's Theorems:

- $(A \cdot B)' = A' + B'$
- $(A + B)' = A' \cdot B'$

9. Consensus Theorem:

- $(A + B)(A + B') = A + B$
- $(A \cdot B) + (A \cdot B') = A$

10. Universal Bound Laws

- $A + A'B = A + B$
- $A \cdot (A' + B) = A \cdot B$

11. Redundancy (Covering) Theorem

- $A + AB = A$
- $A(A + B) = A$

Duality Theorem

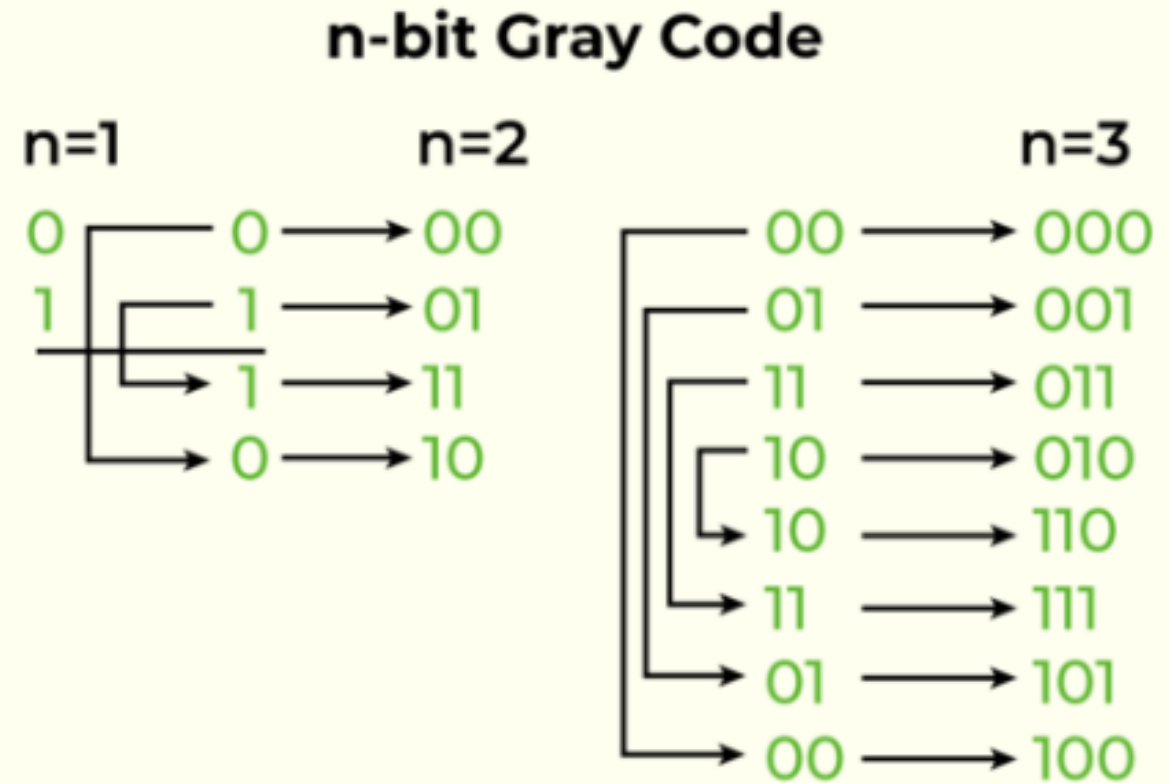
The Duality Theorem states that every Boolean algebra equation remains valid if:

- The AND and OR operators are interchanged.
- The constants 0 and 1 are swapped.

	Function	Dual
Commutation	$A + B = B + A$	$AB + BA$
Association	$A + (B + C) = (A + B) + C$	$A(BC) = (AB)C$
Distribution	$A + BC = (A + B)(A + C)$	$A(B + C) = AB + AC$
Absorption	$A + AB = A$	$A(A + B) = A$
De Morgan	$\overline{A+B} = \bar{A} \cdot \bar{B}$	$\overline{AB} = \bar{A} + \bar{B}$
Consensus	$AC + B\bar{C} =$ $AB + AC + B\bar{C}$	$(A + C)(B + \bar{C}) =$ $(A + B)(A + C)(B + \bar{C})$

Gray code

Gray code is a unit distance code, also known as Reflected Binary Code, is a binary numeral system in which two successive values differ in only one bit. This is in contrast to standard binary representation, where multiple bits might change when transitioning from one value to the next.



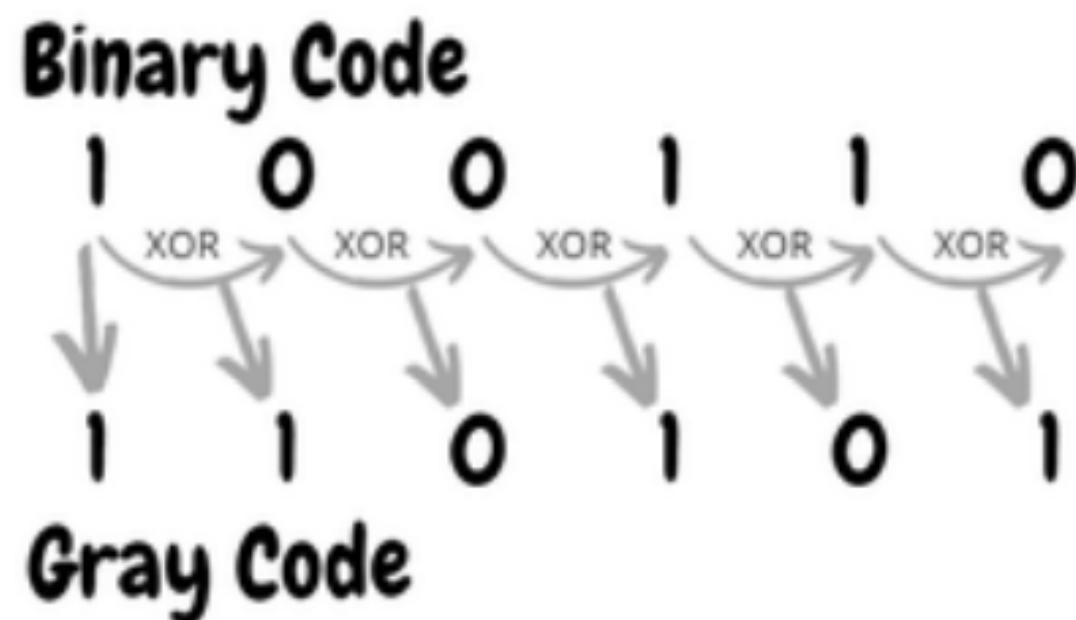
Binary to Gray Code

The most significant bit (MSB) of the binary number is the same as the MSB of the Gray code.

For each subsequent bit:

Binary bit = Previous Binary bit XOR

Current Gray bit.



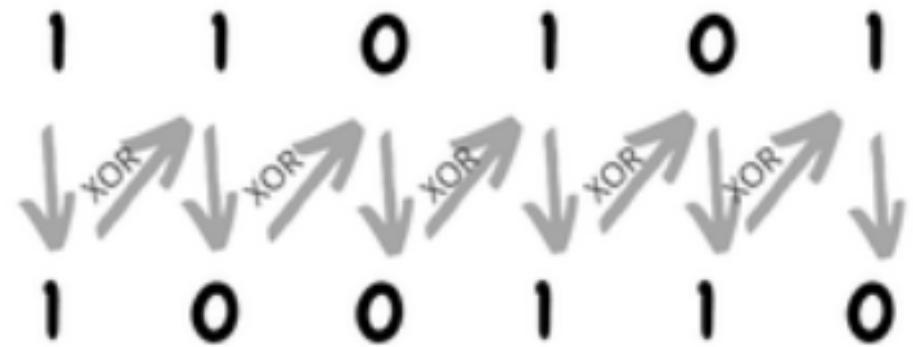
Gray Code to Binary

The MSB of the Binary number is the same as the MSB of the Gray code.

Other Bits:

For each subsequent bit, **XOR** the previous Binary bit with the corresponding Gray bit.

Gray Code



Binary Code

Some common Self complementing code

1. Excess-3 Code

- Each decimal digit is represented in binary form after adding +3.
- Example:
 - $0 \rightarrow 0011$ (3 added to 0)
 - $1 \rightarrow 0100$ (3 added to 1)
 - $9 \rightarrow 1100$ (3 added to 9)

Complement Property:

- The complement of 0011 (for 0) is 1100, which represents 9.

Some common Self complementing code

2. 9's Complement Code

- Each decimal digit d is encoded as $9 - d$.
- Example:
 - $0 \rightarrow 1001$ ($9 - 0$)
 - $1 \rightarrow 1000$ ($9 - 1$)
 - $9 \rightarrow 0000$ ($9 - 9$)

Complement Property:

- 1001 for 0 complements 0000 for 9.

Some common Self complementing code

3. 2-4-2-1 Code

- Weighted binary code with weights 2, 4, 2, 1.
- Example:
 - $0 \rightarrow 0000$
 - $1 \rightarrow 0001$
 - $9 \rightarrow 1001$

Complement Property:

- The code's weights ensure self-complementarity.

Minterms and Maxterms

Variables			Minterms	Maxterms
A	B	C		
0	0	0	$\bar{A}\bar{B}\bar{C} = m_0$	$A + B + C = M_0$
0	0	1	$\bar{A}\bar{B}C = m_1$	$A + B + \bar{C} = M_1$
0	1	0	$\bar{A}B\bar{C} = m_2$	$A + \bar{B} + C = M_2$
0	1	1	$\bar{A}BC = m_3$	$A + \bar{B} + \bar{C} = M_3$
1	0	0	$A\bar{B}\bar{C} = m_4$	$\bar{A} + B + C = M_4$
1	0	1	$A\bar{B}C = m_5$	$\bar{A} + B + \bar{C} = M_5$
1	1	0	$AB\bar{C} = m_6$	$A + B + \bar{C} = M_6$
1	1	1	$ABC = m_7$	$\bar{A} + \bar{B} + \bar{C} = M_7$

Representation of Minterms and Maxterms

$$\begin{aligned} 1. \quad Y &= \overline{A}\overline{B}\overline{C} + \overline{A}B\overline{C} + A\overline{B}C + ABC \\ &= m_0 + m_2 + m_5 + m_7 \\ &= \sum m(0, 2, 5, 7) \end{aligned}$$

$$\begin{aligned} 2. \quad Y &= (A + B + C)(\overline{A} + B + C)(\overline{A} + \overline{B} + \overline{C}) \\ &= M_1 + M_4 + M_7 \\ &= \prod M(1, 4, 7) \end{aligned}$$

where $\sum$ and $\prod$ denote the sum of product and product of sum respectively.

SOP & POS

In SOP or Sum of Product, form variables are ANDed(i.e multiplied) together and used to interpret outputs with logic '1' combination. In SOP form each product term is identified as a minterm.

Example: – $ABC' + A'BC + ABC$
110 011 111

In POS or Product of Sum, form variables are ORed(i.e added) together and applied to represent outputs with logic '0' combination. In POS form every single term is known as the maxterm.

Example: – $(A+B'+C)(A'+B+C')(A+B+C)$
010 101 000

Three-Variable K-map

The three-variable K-map is represented by an array of eight cells (eight minterms or eight maxterms). For understanding purposes, we will use A, B, and C as the three variables. However one can use any letter for the names of the variables. The below figure presents three variables K-Map with SOP and POS representation.

		BC			
		00	01	11	10
A	0	m0	m1	m3	m2
	1	m4	m5	m7	m6

SOP Form

		BC			
		00	01	11	10
A	0	M0	M1	M3	M2
	1	M4	M5	M7	M6

POS Form

Four Variable K-Map

DE		00	01	11	10
BC	00	m0	m1	m3	m2
	01	m4	m5	m7	m6
	11	m12	m13	m15	m14
	10	m8	m9	m11	m10

Five Variable K map

		DE			
		00	01	11	10
BC	00	m0	m1	m3	m2
	01	m4	m5	m7	m6
	11	m12	m13	m15	m14
	10	m8	m9	m11	m10
A=0					

		DE			
		00	01	11	10
BC	00	m16	m17	m19	m18
	01	m20	m21	m23	m22
	11	m28	m29	m31	m30
	10	m24	m25	m27	m26
A=1					

Rules for Grouping

- Groups must contain 1,2,4,8...cells (powers of 2).
- Groups can wrap around the edges of the K-Map.
- Overlapping groups are allowed and sometimes necessary.
- Each group should be as large as possible while including only 1s.

Prime Implicant

Any single 1 or group of 1s in the Karnaugh map of a function F is an implicant of F .

A product term is called a prime implicant of F if it cannot be combined with another term to eliminate a variable.

AB \ CD	CD			
	00	01	11	10
00	1		1	1
01			1	1
11	1			
10	1	1		

- (a) $A'B'C$
- (b) BD
- (c) $A'B'C'D'$
- (d) $A'C$
- (e) $A'B'D'$

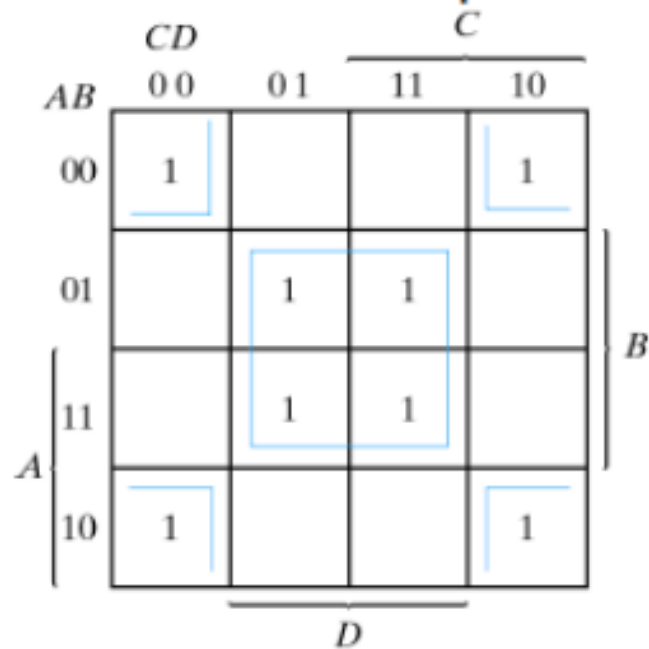
Implicants:
(a),(c),(d),(e)

Prime Implicants:
(d),(e)

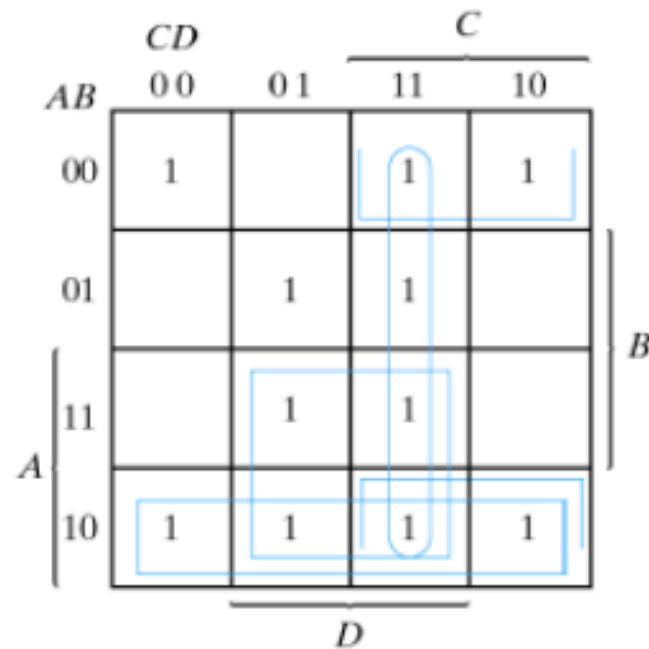
Essential Prime Implicant

A product term is an essential prime implicant if there is a minterm that is only covered by that prime implicant

The minimal sum-of-products form of F must include all the essential prime implicants of F



(a) Essential prime implicants
 BD and $B'D'$



(b) Prime implicants CD , $B'C$
 AD , and AB'

Grouping in K-map

Two variable maps

A \ B	0	1
	0	1
0	0	1
1	1	0

$$F = A\bar{B} + \bar{A}B$$

A \ B	0	1
	0	1
0	0	1
1	1	1

$$F = AB + \bar{A}B + A\bar{B}$$
$$F = A + B$$

Three variable maps

A \ BC	00	01	11	10
	0	1	0	1
0	0	1	0	1
1	1	1	1	1

$$F = A + \bar{B}\bar{C} + B\bar{C}$$

$$F = \bar{A}\bar{B}\bar{C} + \bar{A}\bar{B}C + A\bar{B}\bar{C} + A\bar{B}C + \bar{A}B\bar{C} + \bar{A}BC$$

Grouping in K-map

AB \ CD		CD			
		00	01	11	10
00	00	1 0	1 1	0 3	1 2
	01	1 4	0 5	0 7	1 6
11	11	0 12	0 13	1 15	1 14
	10	0 8	1 9	0 11	0 10

$B_3B_2 \backslash B_1B_0$		B_1B_0			
		00	01	11	10
00	0	0	1	1	
01	1	1	0	0	
11	1	1	0	0	
10	0	0	1	1	

Don't Care Condition

In some practical logic designs, there are combinations of input variable that we know will never occur.

These combinations are marked as 'd' or 'x'.

In k map, these can be mapped treating as '1' to form larger groups or can be disregarded treating as '0'.

Since it doesn't matter whether to be treated as '1' or '0', these are called don't care.

Don't Care Condition

An 8421 binary coded decimal is a 4 bit coding system. Codes from 0000-1001 are used to represent numbers from 0 to 9. Remaining 6 unused combinations 1010-1111 are don't care conditions.

Decimal	Binay (BCD)			
	8	4	2	1
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Don't Care Condition

In some cases, outputs are undefined

We “don't care” if the circuit produces a ‘0’ or a ‘1’

This knowledge can be used to simplify functions

		A			
		00	01	11	10
CD	00	0	0	X	0
	01	1	1	X	1
	11	1	1	0	0
	10	0	X	0	0

Vertical labels: **C** on the left, **D** on the right, **B** at the bottom.

- Treat X's like either 1's or 0's
- Very useful
- OK to leave some X's uncovered

Variable Entered Map

Variable entered mapping (VEM) is used to reduce the size of K-maps with many variables.

In VEM, one variable is chosen as the map entered variable and included in the K-map along with zeros, ones, and don't cares.

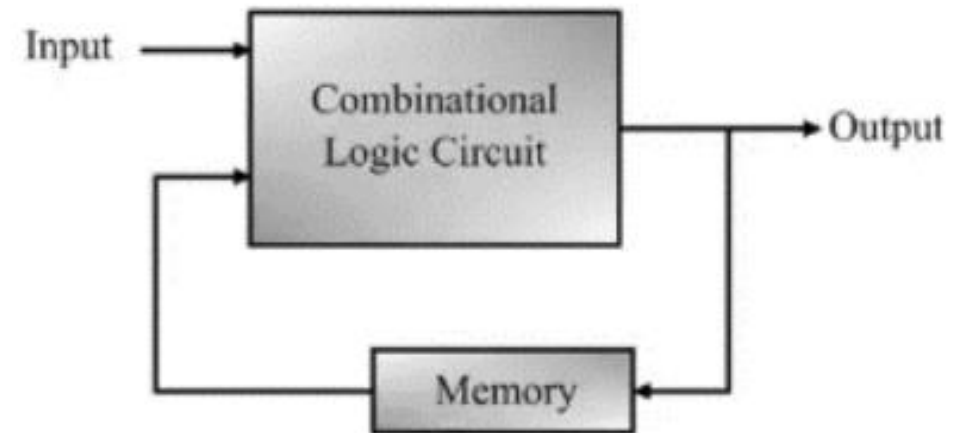
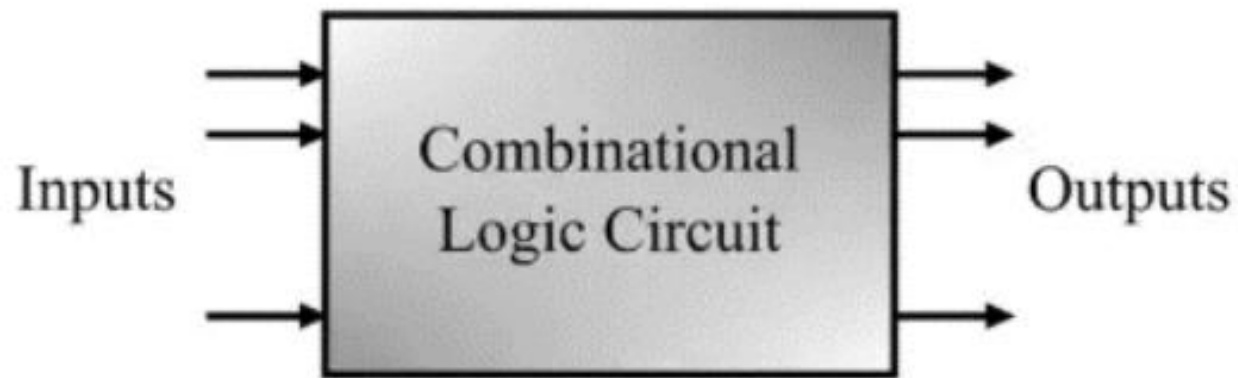
This allows representing functions with more variables using a smaller K-map.

A	B	C	D	F	
0	0	0	0	0	0
0	0	0	1	0	
0	0	1	0	1	1
0	0	1	1	1	
0	1	0	0	0	D
0	1	0	1	1	
0	1	1	0	0	D
0	1	1	1	1	
1	0	0	0	0	0
1	0	0	1	0	
1	0	1	0	X	X
1	0	1	1	X	
1	1	0	0	1	D'
1	1	0	1	0	
1	1	1	0	1	D'
1	1	1	1	0	

		BC			
		00	01	11	10
A	0				
	1				

Digital Circuits

Combinational vs Sequential circuits



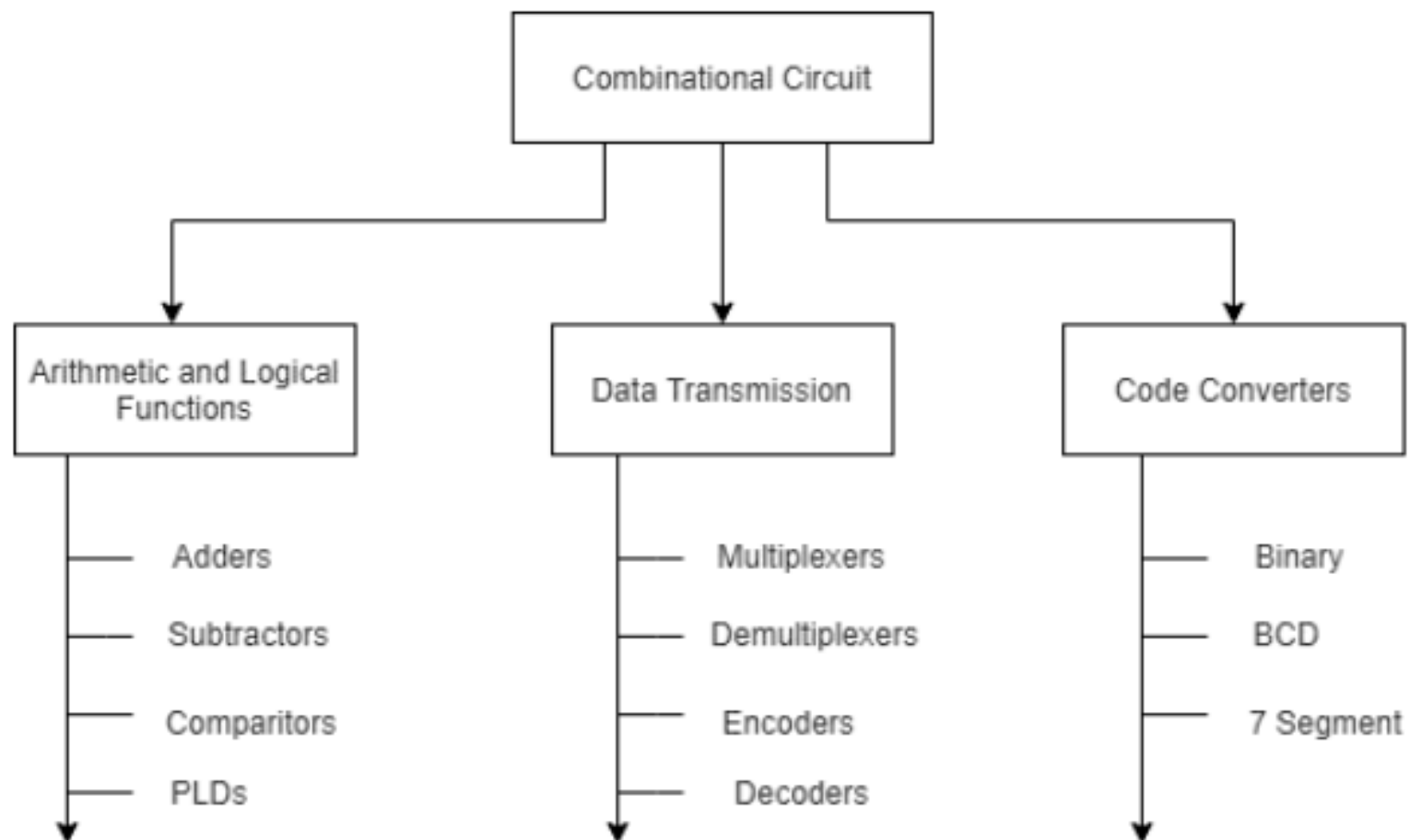
Aspects	Combinational Circuit	Sequential Circuit
Definition	Output depends only on present inputs.	Output depends on present inputs and past states.
Mathematical Model	Boolean expressions (pure logic functions).	Finite State Machine (FSM) with transition functions.
Memory Element	No memory; does not retain past states.	Requires memory elements like flip-flops.
Clock Dependency	Operates asynchronously.	Operates synchronously with a clock signal.
Feedback Loop	No feedback path; purely feedforward logic.	Has feedback paths that influence future outputs.
Power Consumption	Lower, as it operates only when inputs change.	Higher due to continuous clocking and state retention.

Delay Considerations	Propagation delay depends on gate delays only.	Propagation delay depends on clock cycle time and flip-flop setup/hold times.
Design Complexity	Less complex; only involves logic gates.	More complex due to state transitions and memory elements.
Reliability & Noise Sensitivity	More reliable since it does not rely on previous states.	More sensitive to noise due to clock and storage elements.
Design Techniques	Boolean algebra, Karnaugh maps (K-maps).	State diagrams, state tables, flip-flop excitation tables.
Examples	ALU (Arithmetic Logic Unit), Encoders, Decoders, Adders, Multiplexers.	Counters, Shift Registers, Latches, Flip-Flops, RAM.

Difference Summary

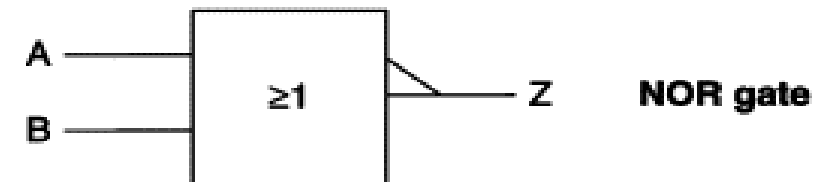
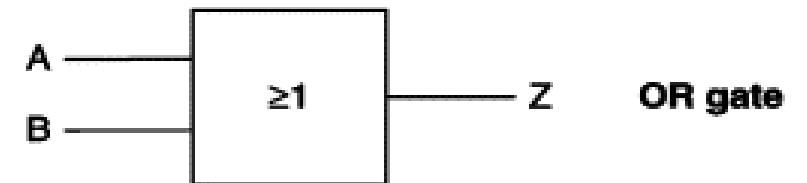
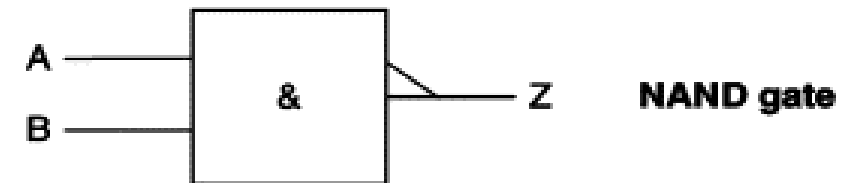
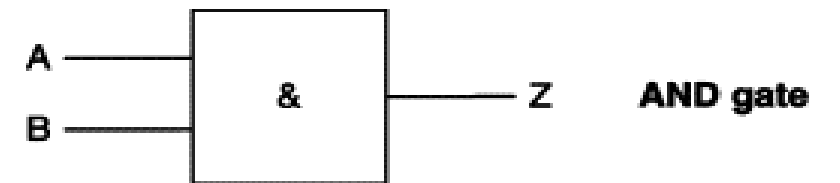
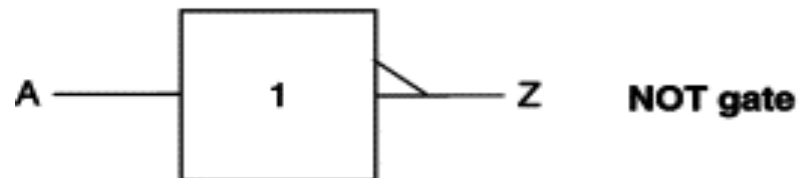
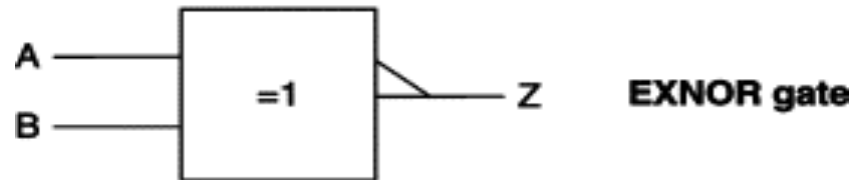
Feature	Combinational Circuits	Sequential Circuits
Memory	No memory	Requires memory
Output	Depends on inputs	Depends on inputs + past states
Examples	Adders, Encoders	Flip-flops, Counters

Classification



Combinational Logic Circuits

An alternative set of logic symbols, IEEE/ANSI standard 91-1984 (Graphics Symbols for Logic Functions), is shown here:



Universal Property of NAND

1- The NAND Gate as a Universal Logic Element

NAND Definition: A NAND gate is an AND gate followed by a NOT gate.

Truth Table:

A	B	A NAND B
0	0	1
0	1	1
1	0	1
1	1	0

Basic Implementations:

- NOT using NAND: $A \text{ NAND } A = \text{NOT } A$
- AND using NAND: $(A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$
- OR using NAND: $(A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$

Universal Property of NAND

1- The NAND Gate as a Universal Logic Element

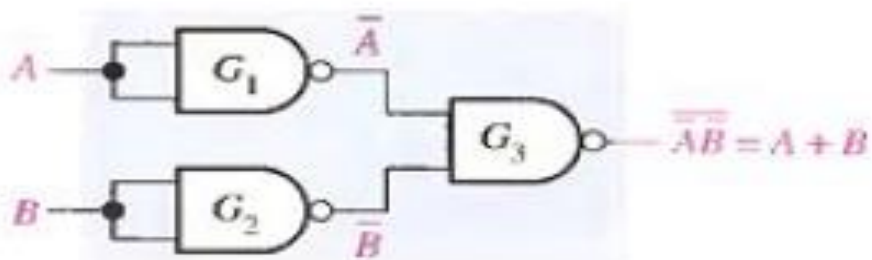
The NAND gate is a universal gate because it can be used to produce the NOT, the AND, the OR, and the NOR functions. An inverter can be made from a NAND gate by connecting all of the inputs together and creating, in effect, a single input, as shown in Fig. (a) for a 2-input gate. An AND function can be generated by the use of NAND gates alone, as shown in Fig.(b). An OR function can be produced with only NAND gates, as illustrated in part (c). Finally, a NOR function is produced as shown in part (d).



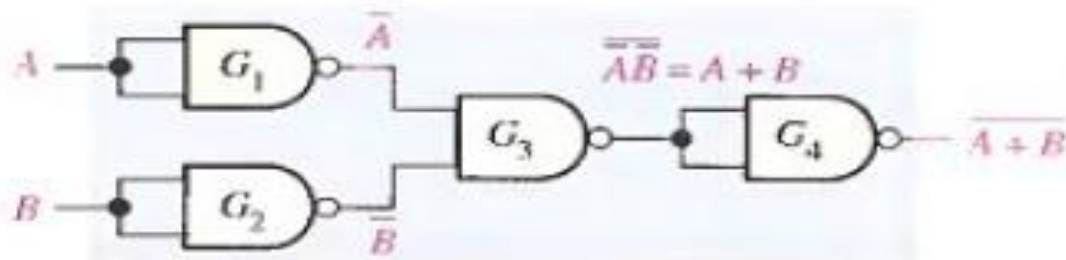
(a) One NAND gate used as an inverter



(b) Two NAND gates used as an AND gate



(c) Three NAND gates used as an OR gate



(d) Four NAND gates used as a NOR gate

Universal Property of NOR

2- The NOR Gate as a Universal Logic Element

NOR Definition: A NOR gate is an OR gate followed by a NOT gate.

Basic Implementations:

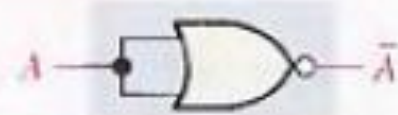
- NOT using NOR: $A \text{ NOR } A = \text{NOT } A$
- OR using NOR: $(A \text{ NOR } B) \text{ NOR } (A \text{ NOR } B)$
- AND using NOR: $(A \text{ NOR } A) \text{ NOR } (B \text{ NOR } B)$

A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

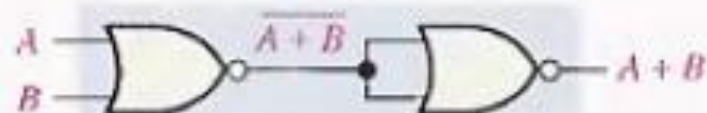
Universal Property of NOR

2- The NOR Gate as a Universal Logic Element

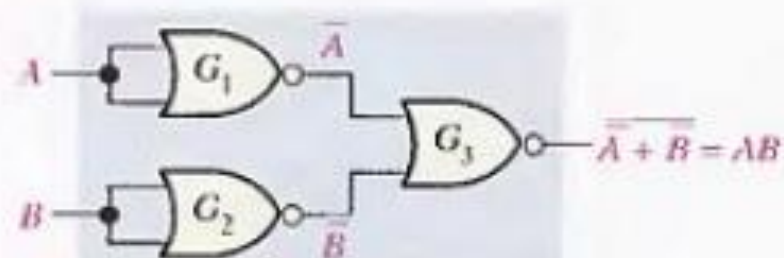
Like the NAND gate, the NOR gate can be used to produce the NOT, AND, OR and NAND functions. A NOT circuit, or inverter, can be made from a NOR gate by connecting all of the inputs together to effectively create a single input, as shown in Fig.(a) with a 2-input example. Also, an OR gate can be produced from NOR gates, as illustrated in Fig.(b). An AND gate can be constructed by the use of NOR gates, as shown in Fig.(c). In this case the NOR gates G 1 and G 2 are used as inverters. Fig (d) shows how NOR gates are used to form NAND function.



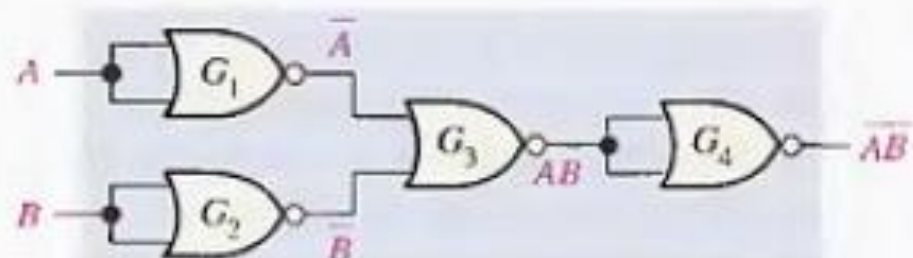
(a) One NOR gate used as an inverter



(b) Two NOR gates used as an OR gate



(c) Three NOR gates used as an AND gate



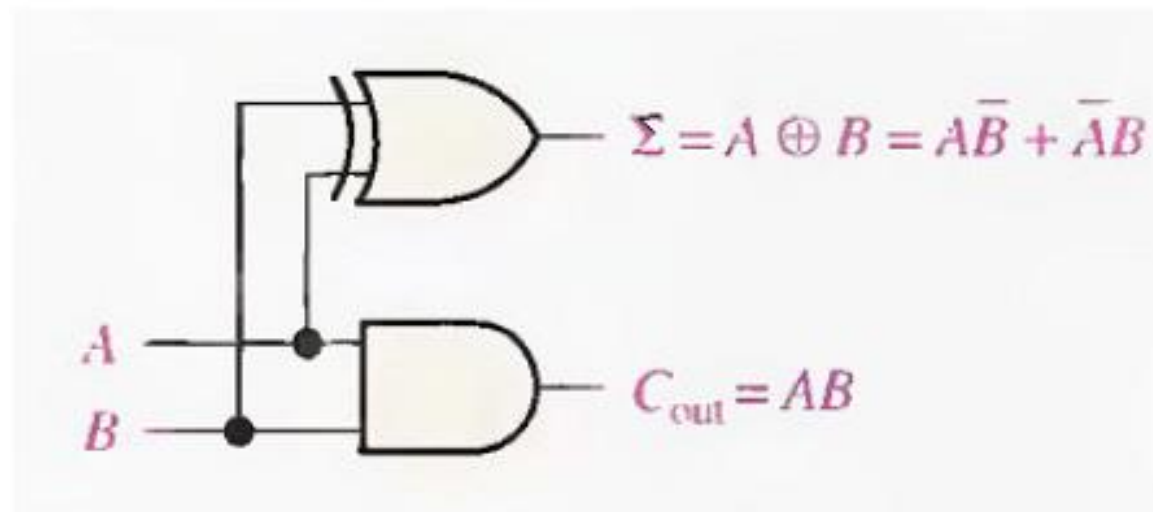
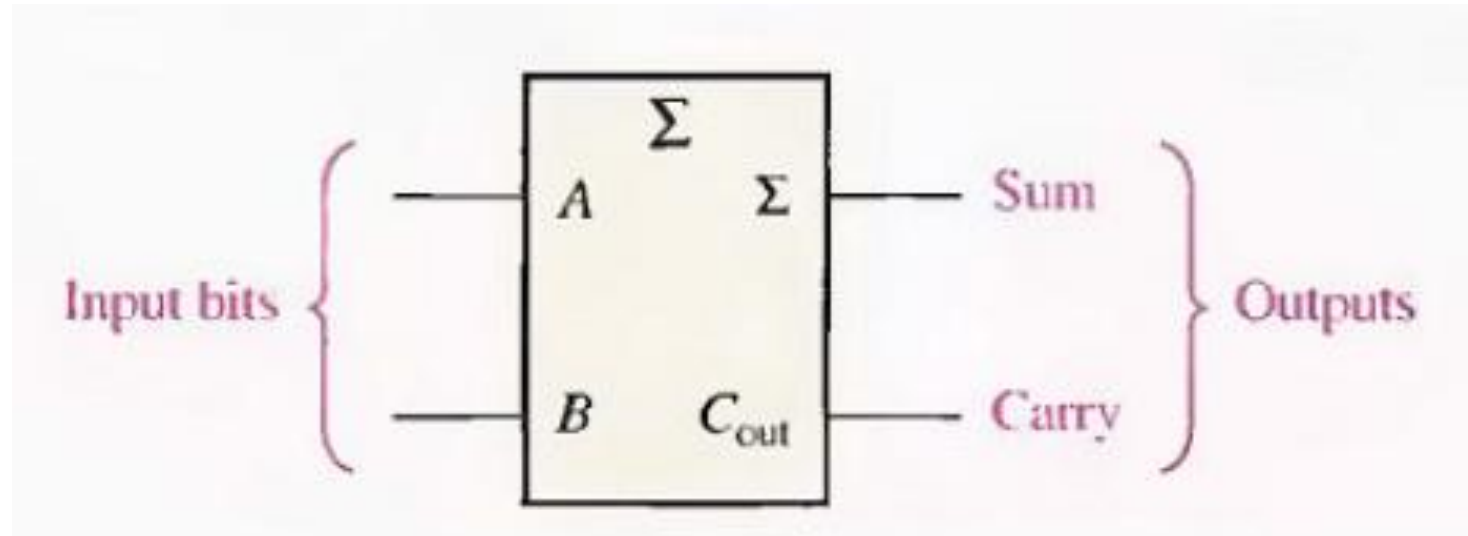
(d) Four NOR gates used as a NAND gate

NAND and NOR implementation

Feature	NAND Implementation	NOR Implementation
Number of Gates	Generally fewer	Slightly more
Speed	Faster in some cases	May introduce delay
Common Usage	Widely used in microprocessors	Used in memory design

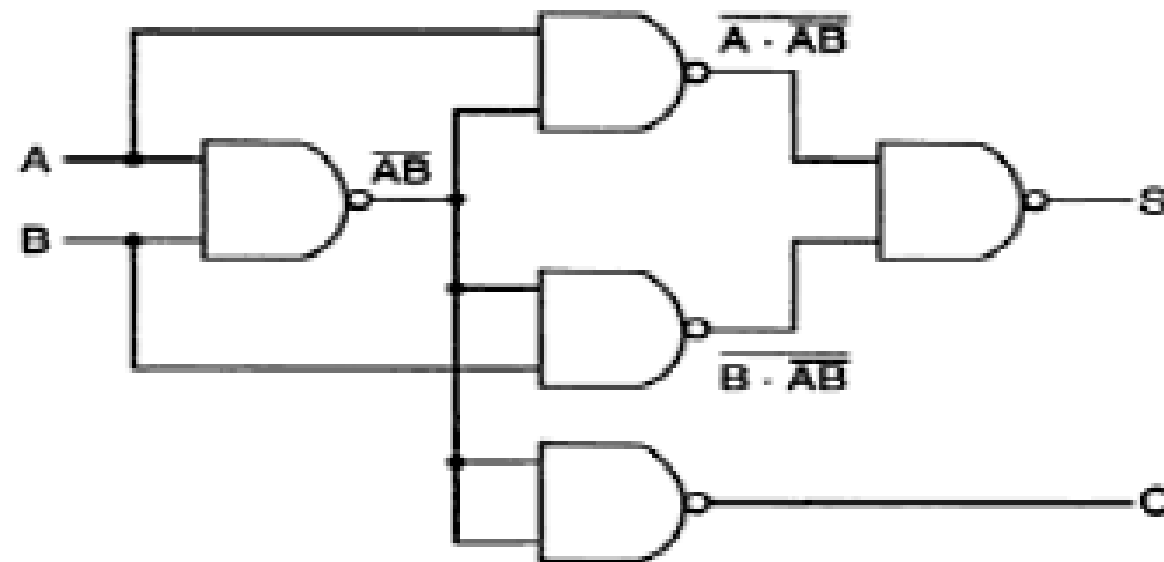
BASIC ADDERS

The Half-Adder



Half Adder

$$\begin{aligned} S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + B) + B(\bar{A} + \bar{B}) \\ &= A \cdot \overline{AB} + B \cdot \overline{AB} \\ &= \overline{A \cdot AB \cdot B \cdot AB} \\ C &= AB = \overline{\overline{AB}} \end{aligned}$$



Logic diagram of a half-adder using only 2-input NAND gates.

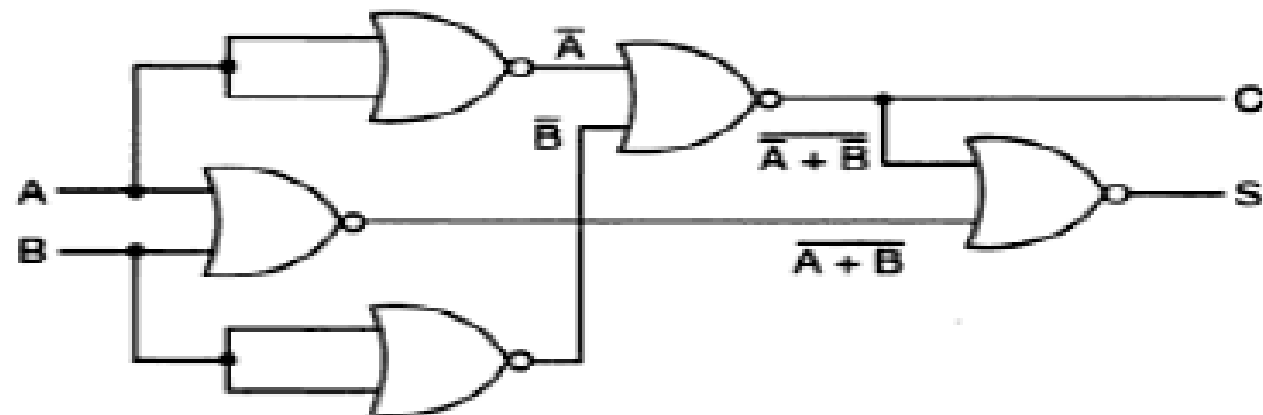
Half Adder

$$\begin{aligned} S &= A\bar{B} + \bar{A}B = A\bar{B} + A\bar{A} + \bar{A}B + B\bar{B} \\ &= A(\bar{A} + B) + B(\bar{A} + \bar{B}) \end{aligned}$$

$$= (A + B)(\bar{A} + \bar{B})$$

$$= \overline{\overline{A + B} + \overline{\bar{A} + \bar{B}}}$$

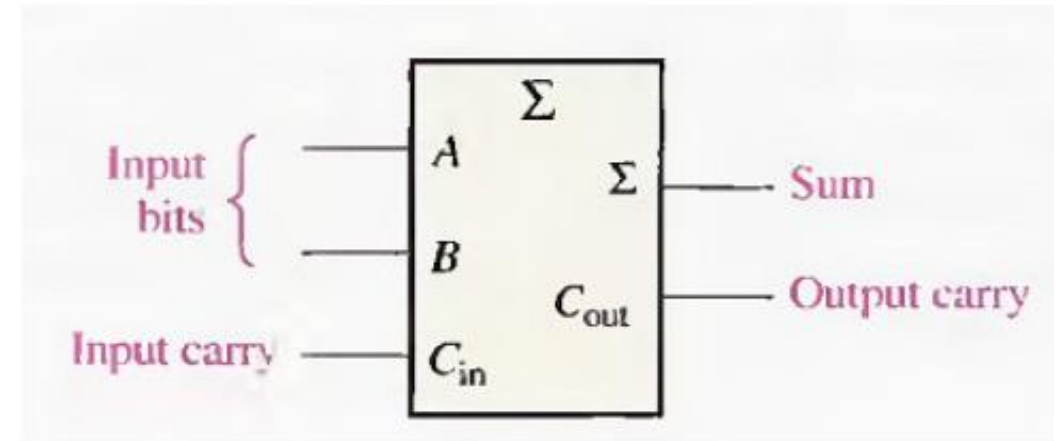
$$C = AB = \overline{\overline{AB}} = \overline{\overline{A} + \overline{B}}$$



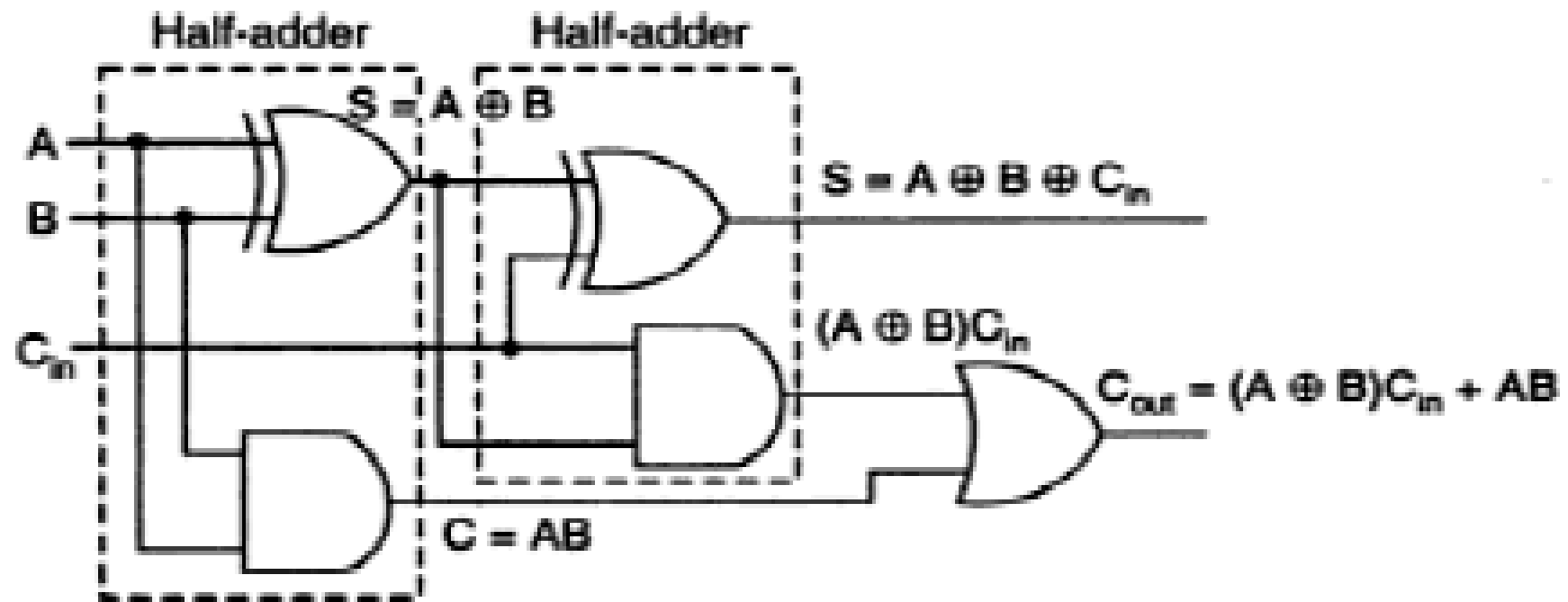
Logic diagram of a half-adder using only 2-input NOR gates.

Full Adder

A	B	C_{in}	C_{out}	Σ
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

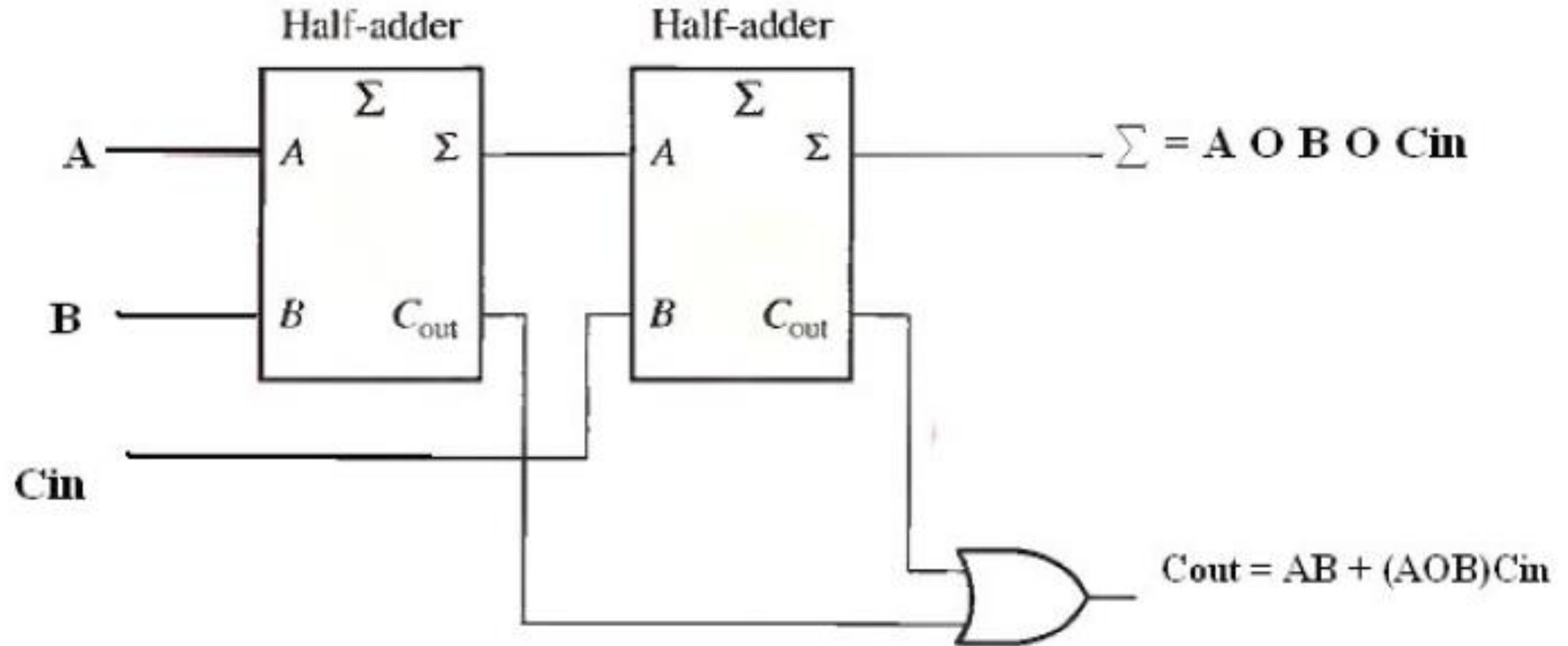


Two half adder to make a Full Adder



Logic diagram of a full-adder using two half-adders.

Two half adder to make a Full Adder

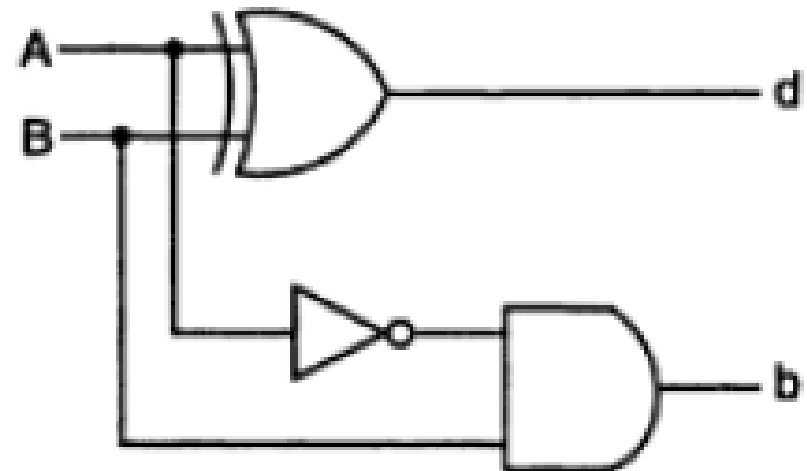


Half Subtractor

A Half-subtractor is a combinational circuit with two inputs A and B and two outputs d and b. d indicates the difference and b is the output signal generated that informs the next stage that a 1 has been borrowed. When a bit B is subtracted from another bit A, a difference bit (d) and a borrow bit (b) result according to the rules given as,

Inputs		Outputs	
A	B	d	b
0	0	0	0
1	0	1	0
1	1	0	0
0	1	1	1

(a) Truth table

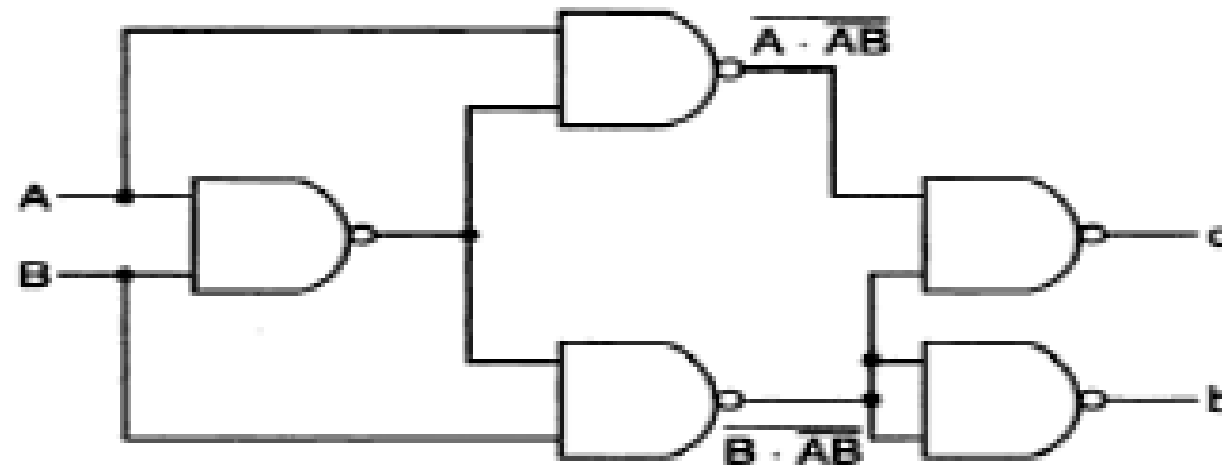


Half Subtractor

NAND Logic:

$$d = A \oplus B = \overline{\overline{A \cdot \overline{AB}} \cdot B \cdot \overline{AB}}$$

$$b = \overline{A}B = B(\overline{A} + \overline{B}) = B(\overline{AB}) = \overline{B \cdot \overline{AB}}$$

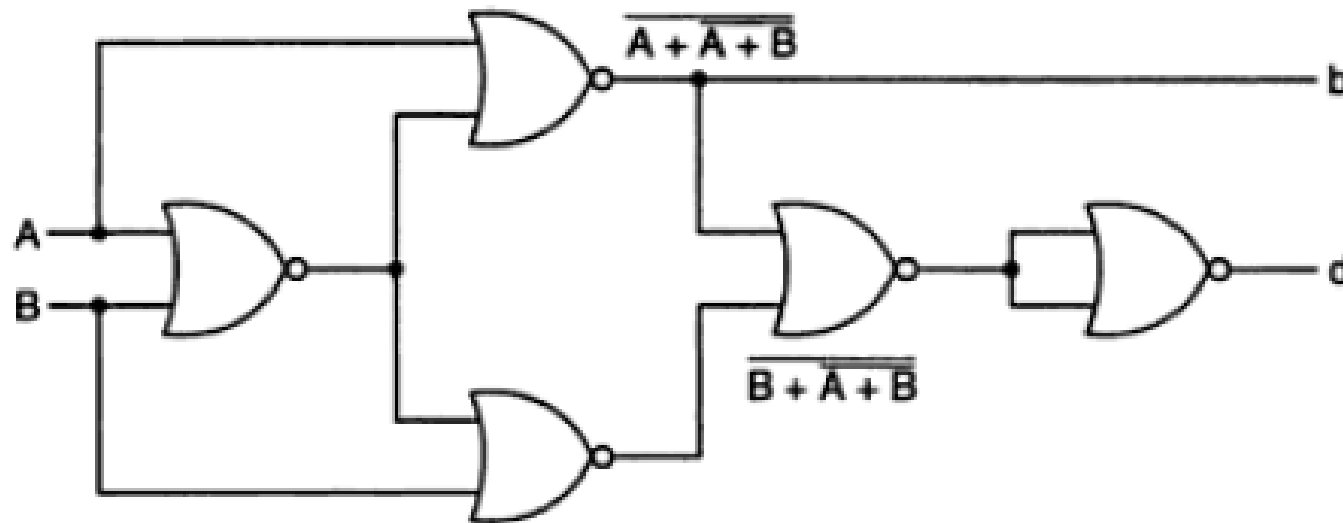


Logic diagram of a half-subtractor using only 2-input NAND gates.

Half Subtractor

NOR Logic:

$$\begin{aligned}d &= A \oplus B = A\bar{B} + \bar{A}B = A\bar{B} + B\bar{B} + \bar{A}B + A\bar{A} \\&= \bar{B}(A + B) + \bar{A}(A + B) = \overline{\overline{B} + \overline{A + B}} + \overline{\overline{A} + \overline{A + B}} \\d &= \bar{A}B = \bar{A}(A + B) = \overline{\overline{\bar{A}(A + B)}} = \overline{A + (\bar{A} + \bar{B})}\end{aligned}$$

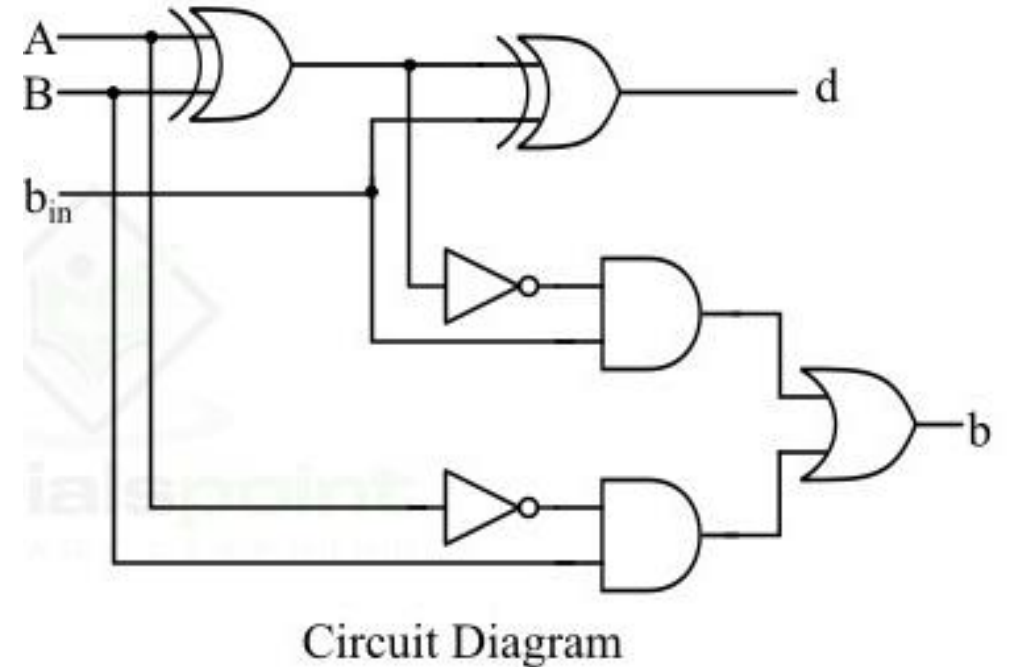
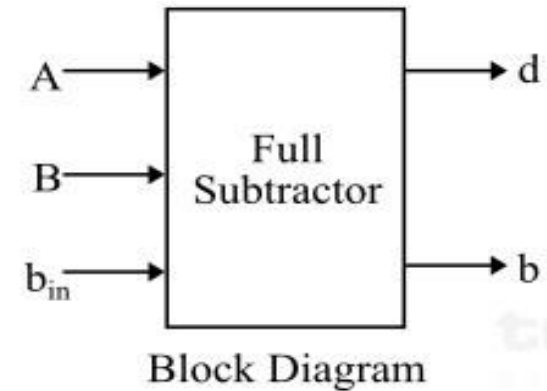


Logic diagram of a half-subtractor using only 2-input NOR gates.

Full Subtractor

Inputs			Difference	Borrow
A	B	b_{in}	d	b
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

(a) Truth table



Parallel Adders

Types of Parallel Adders

Ripple Carry Adder (RCA)

Simple structure, but slow due to carry propagation delay.

Carry Look-Ahead Adder (CLA)

Faster by reducing carry propagation delay.

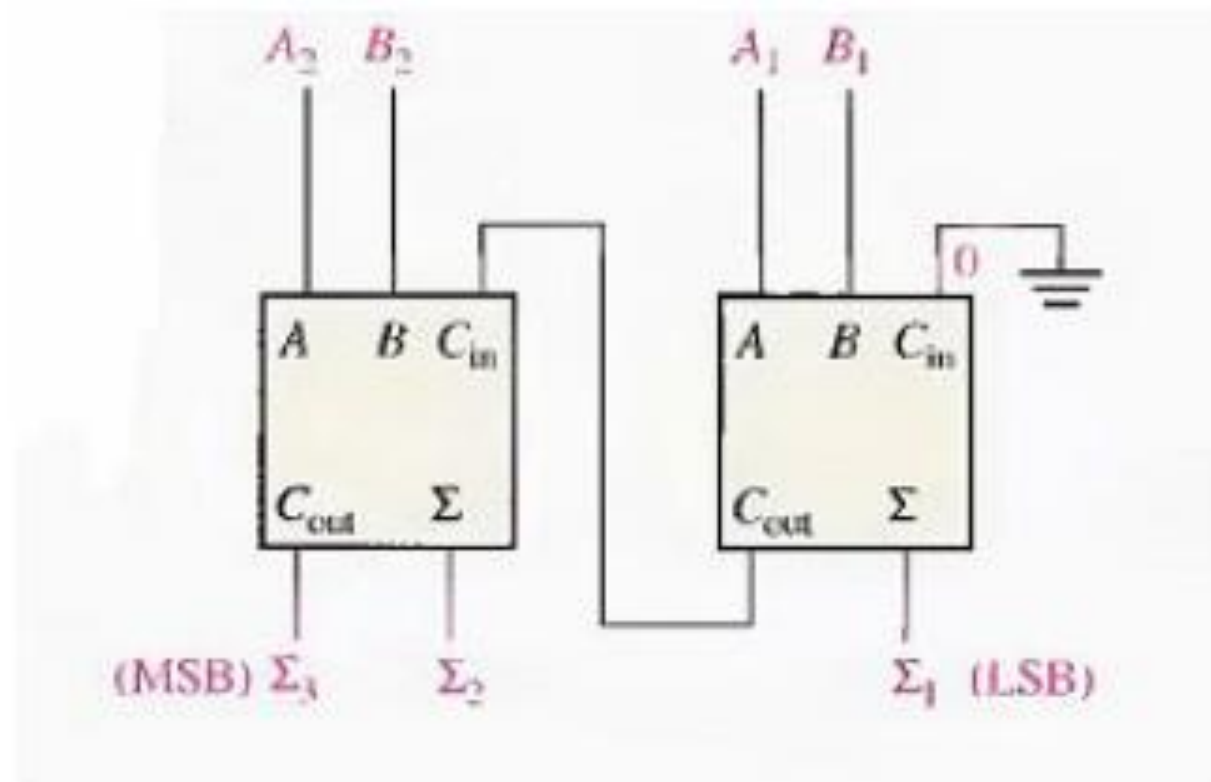
Carry Save Adder (CSA)

Used for high-speed multiplication operations.

Parallel Adders

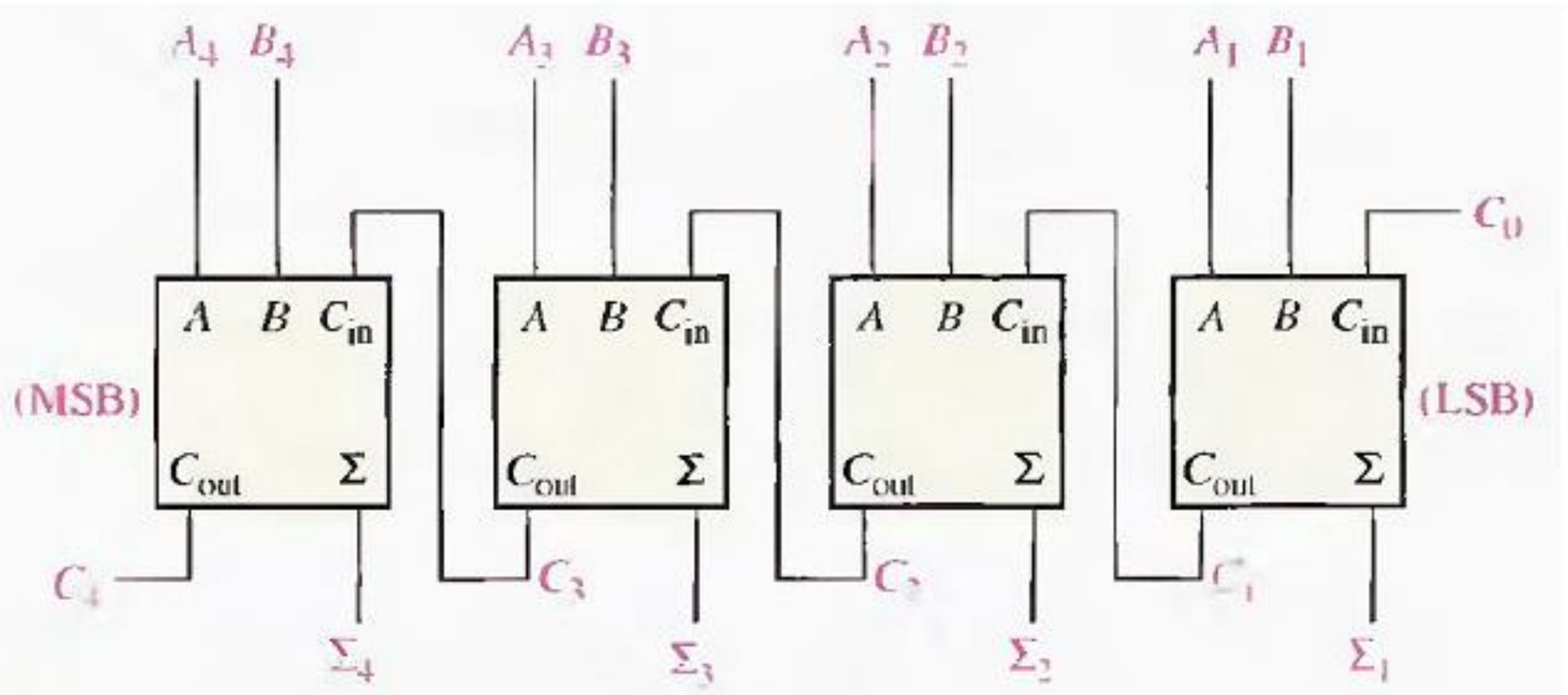
Type	Speed	Structure Complexity	Application
Ripple Carry Adder	Slow	Simple	Small circuits
Carry Look-Ahead Adder	Fast	Moderate	ALUs, Processors
Carry Save Adder	Fastest	Complex	Multipliers

2 Bit Parallel Adders

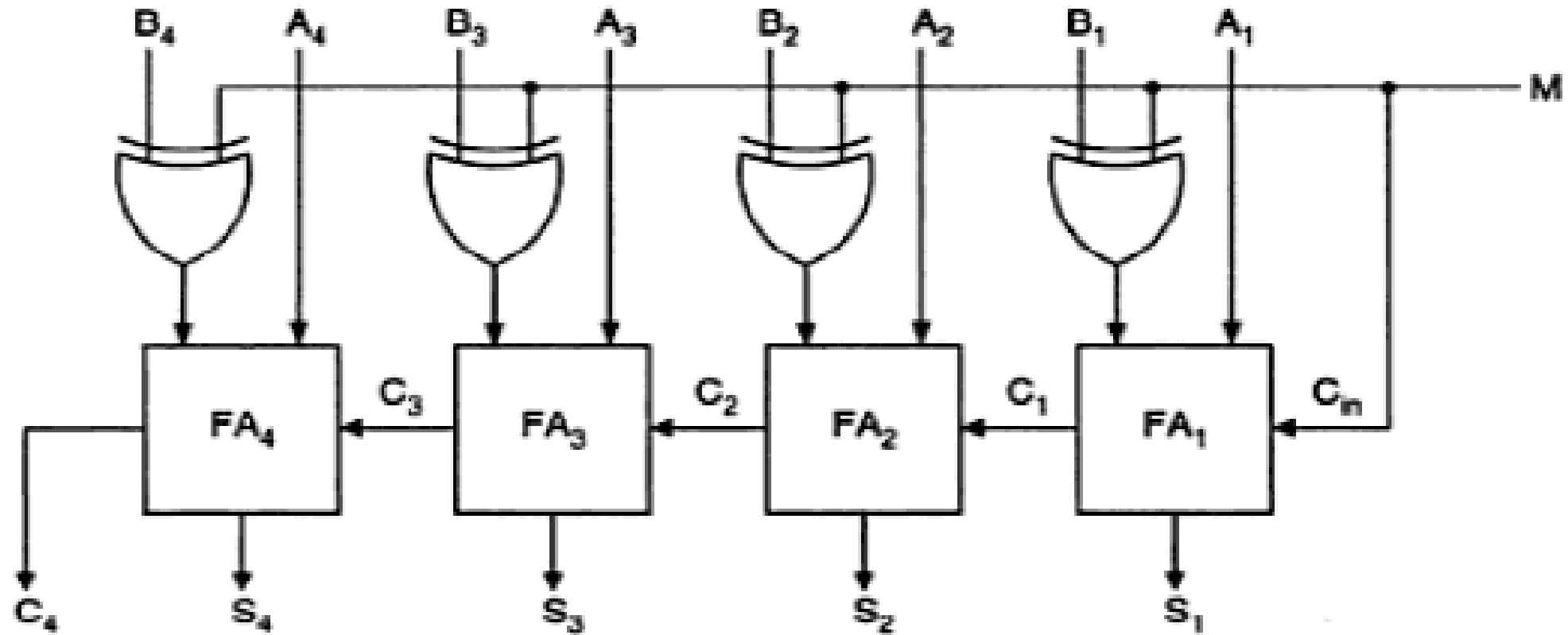


Block diagram of a basic 2-bit parallel adder using two full-adders.

4 Bit Parallel Adders



Adder-Subtractor



Logic diagram of a 4-bit binary adder-subtractor.

Thank you!